

國立陽明交通大學  
電機資訊國際學位學程  
碩士論文

EECS International Graduate Program  
National Yang Ming Chiao Tung University  
Master Thesis

分散式人工智慧代理驅動框架  
協同威脅偵測與回應

Distributed AI Agent-driven Framework  
for Coordinated Threat Detection and Response

研 究 生：莊輝柱（Trang Huy Tru）

指導教授：林盈達（Ying-Dar Lin）

中華民國一一五年六月

June 2026

分散式人工智慧代理驅動框架  
協同威脅偵測與回應  
Distributed AI Agent-driven Framework  
for Coordinated Threat Detection and Response

研 究 生：莊輝柱  
指導教授：林盈達 博士

Student: Trang Huy Tru  
Advisor: Dr. Ying-Dar Lin

國立陽明交通大學  
電機資訊國際學位學程  
碩士論文  
A Thesis  
Submitted to EECS International Graduate Program  
College of Electrical Engineering and Computer Science  
National Yang Ming Chiao Tung University  
in Partial Fulfilment of the Requirements  
for the Degree of  
Master in  
Electrical Engineering and Computer Science

June 2026  
Taiwan, Republic of China

中華民國 一一五年六月

# 分散式人工智慧代理驅動框架 協同威脅偵測與回應

學生：莊輝柱

指導教授：林盈達 博士

國立陽明交通大學 電機資訊國際學位學程

## 摘 要

現代網路攻擊通常具有多階段特性，並會在 EDR、NDR、IDS/IPS 與防火牆等異質資安工具中留下零散的攻擊證據。傳統以安全營運中心（SOC）為核心的架構會集中蒐集並分析警報，但此設計依賴完整證據彙整，容易造成警報過載、通訊延遲與較慢的回應速度。為解決上述限制，本論文提出一個分散式 AI Agent 驅動的威脅偵測與回應框架，將 AI Agent 部署於各資安工具中，使其能進行本地偵測、情境共享、攻擊生命週期重建與回應決策。此框架使用 MITRE ATT&CK 將異質警報正規化為攻擊技術，並採用輕量化 TextCNN 模型，從結構化與半結構化資安警報中辨識攻擊技術。偵測到的技術會根據時間戳與 MITRE tactic 順序進行關聯分析，而強化學習回應模組則在攻擊阻擋效果與系統操作成本之間取得平衡，以選擇適當防禦動作。本研究在受控測試環境中實作此框架，使用 Wazuh、Suricata、OPNsense、NATS、Redis 與 Docker 化 Agents。實驗結果顯示，分散式框架在 1.0 秒觀察視窗下可達到與集中式基準相近的偵測準確率 0.903，同時將偵測時間由 5.250 秒降低至 2.175 秒，延遲減少 58.6%。在回應方面，分散式框架能比集中式基準更早停止攻擊生命週期，將平均停止步驟由約 4.0 降至約 2.5，約提前 1.5 個步驟，並將回應時間由 13.60 秒縮短至 9.22 秒，提升幅度達 47.51%。整體而言，本論文證明將智慧從集中式 SOC 移至分散式 AI Agents，可提升網路威脅偵測與回應的速度、協調能力與可擴展性。

**關鍵詞：**分散式 AI Agent, 多 Agent 系統, 偵測與回應

# **Distributed AI Agent-driven Framework for Coordinated Threat Detection and Response**

Student: Trang Huy Tru

Advisor: Dr. Ying-Dar Lin

EECS International Graduate Program  
National Yang Ming Chiao Tung University

## **Abstract**

Modern cyberattacks are often multi-stage and leave fragmented evidence across heterogeneous security tools such as EDR, NDR, IDS/IPS, and firewalls. Traditional SOC-based architectures centralize alert collection and analysis, but this design depends on complete evidence aggregation and may introduce alert overload, communication delay, and slower response. To address these limitations, this thesis proposes a distributed AI agent-driven framework in which each security tool is equipped with an AI agent for local detection, context sharing, lifecycle reconstruction, and response decision-making. MITRE ATT&CK is used to normalize alerts into attack techniques, while lightweight TextCNN models detect techniques from structured and semi-structured security alerts. Detected techniques are correlated using timestamps and MITRE tactic ordering, and a reinforcement learning-based response module selects actions by balancing blocking effectiveness and operational cost. The framework is implemented in a controlled testbed using Wazuh, Suricata, OPNsense, NATS, Redis, and Docker-based agents. Experimental results show that the distributed framework achieves detection accuracy comparable to the centralized baseline, reaching 0.903 accuracy at a 1.0-second window, while reducing detection time from 5.250 seconds to 2.175 seconds, a 58.6% latency reduction. For response, the distributed framework stops attack lifecycles approximately 1.5 steps earlier, reducing the average stopping step from about 4.0 to 2.5, and shortens response time from 13.60 seconds to 9.22 seconds, achieving a 47.51% improvement. Overall, this thesis demonstrates that moving intelligence from a centralized SOC to distributed AI agents improves the speed, coordination, and scalability of cyber threat detection and response.

**Keywords: Distributed AI Agent, Multi-Agent System, Detection and Response**

# Acknowledgement

回顧這段人生旅程，我滿懷感激之情。這篇論文的完成從來不是一條輕鬆的道路，途中充滿了疑惑、不眠之夜，以及幾度幾乎想要放棄的時刻。然而，在這些困難的時刻推動我繼續前行的，是他人的支持與善意。

首先，我要向我的指導教授林盈達老師表達我最深的感謝。您對「為什麼」的不斷追問，不僅塑造了我研究的邏輯，更深刻地影響了我面對問題的思考方式。您教會我如何更精確、更有耐心地探索問題，並保有好奇心。這些學習將超越論文本身，伴隨我未來的道路。我也誠摯感謝賴源正老師，黃仁竑老師，與 Didik Sudyana 老師，您們在定期討論中所提供的指導與建議，讓我不斷改進研究內容，並體會到學術清晰與嚴謹的重要性。

感謝 701 MIRC 實驗室的所有夥伴們——您們不僅是同事，更是陪我一起走過這段旅程的重要存在。我要特別感謝學長。您們的指導、見解與正能量對我意義非凡。也謝謝所有的室友與學弟妹們，感謝你們的歡笑、討論，以及那份共同堅持的心。這些回憶我將永遠珍藏於心中。

對於我的家人，我感激不盡。儘管我們遠隔重洋，但你們每週的電話和堅定不移的支持始終是我最堅實的依靠。每一個鼓勵的訊息都恰到好處地出現在我最需要的時候，給予我繼續前進的力量。我也同樣感激世界各地的摯友。我深深地想念你們，雖然我們相隔遙遠，但你們的支持始終讓我感受到，在這條路上我並不孤單。

最後，我要感謝我自己。這是一段充滿挑戰的旅程——學術上的、情感上的和個人方面的。我無數個夜晚都待在實驗室和房間裡，靠著咖啡和內心的堅定信念堅持下去。尤其是在第一年，我無數次認真地想過放棄。但我沒有。我選擇了堅持，最終完成了我開始的事情。

這篇論文不僅是研究成果的體現，更是人與人之間情感、成長與韌性的結晶。我將你們的名字一一珍藏於此篇頁之中。

# Table of Contents

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| 摘要 . . . . .                                                                             | i         |
| Abstract . . . . .                                                                       | ii        |
| Acknowledgement . . . . .                                                                | iii       |
| Table of Contents . . . . .                                                              | iv        |
| List of Figures . . . . .                                                                | vi        |
| List of Tables . . . . .                                                                 | vii       |
| <b>1 Introduction . . . . .</b>                                                          | <b>1</b>  |
| <b>2 Background and Related Works . . . . .</b>                                          | <b>4</b>  |
| 2.1 Security Solutions . . . . .                                                         | 4         |
| 2.2 Security Operation Center (SOC) . . . . .                                            | 5         |
| 2.3 MITRE ATT&CK and Attack Chain Representation . . . . .                               | 6         |
| 2.4 AI-Driven Detection and Response in Cybersecurity . . . . .                          | 7         |
| 2.5 Related Works on AI-Driven and Decentralized Threat Detection and Response . . . . . | 9         |
| <b>3 Problem Formulation . . . . .</b>                                                   | <b>12</b> |
| 3.1 Notations . . . . .                                                                  | 12        |
| 3.2 Problem Statement . . . . .                                                          | 14        |
| <b>4 Distributed AI Agent-Driven Framework for Detection and Response . . . . .</b>      | <b>18</b> |
| 4.1 Solution Overview . . . . .                                                          | 18        |
| 4.2 Communication Module . . . . .                                                       | 20        |
| 4.3 Detection Module . . . . .                                                           | 24        |
| 4.4 Response Module . . . . .                                                            | 25        |
| 4.5 Example Run . . . . .                                                                | 29        |
| <b>5 Implementation . . . . .</b>                                                        | <b>31</b> |
| 5.1 Overall Pipeline . . . . .                                                           | 31        |
| 5.2 Testbed Setup and Data Collection . . . . .                                          | 33        |
| 5.3 Model Training and Integration . . . . .                                             | 35        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 5.4      | Docker Deployment and Communication Setup . . . . . | 35        |
| <b>6</b> | <b>Evaluation and Results . . . . .</b>             | <b>37</b> |
| 6.1      | Experiment Setup . . . . .                          | 37        |
| 6.2      | Detection: Distributed vs. Centralized . . . . .    | 39        |
| 6.3      | Detection Coverage of Each AI Agent . . . . .       | 43        |
| 6.4      | Response Performance: Efficiency and Cost . . . . . | 46        |
| 6.5      | Response Behavior of Each Agent . . . . .           | 48        |
| <b>7</b> | <b>Conclusion and Future Work . . . . .</b>         | <b>50</b> |
|          | <b>References . . . . .</b>                         | <b>52</b> |



# List of Figures

|     |                                                                                       |    |
|-----|---------------------------------------------------------------------------------------|----|
| 2.1 | MITRE ATT&CK matrix in enterprise network . . . . .                                   | 7  |
| 2.2 | The agent-environment interaction in RL . . . . .                                     | 9  |
| 3.1 | Detection and response with distributed AI agents . . . . .                           | 15 |
| 4.1 | Overall framework and AI Agent architecture . . . . .                                 | 19 |
| 4.2 | Communication flow with leader mechanism . . . . .                                    | 21 |
| 4.3 | Response waiting time . . . . .                                                       | 23 |
| 4.4 | Leader role conflict handling . . . . .                                               | 23 |
| 4.5 | TextCNN-based Detection Module . . . . .                                              | 24 |
| 4.6 | Reinforcement learning-based Response Module . . . . .                                | 25 |
| 4.7 | Example run of the proposed distributed detection and response process . . . . .      | 29 |
| 5.1 | Implementation architecture . . . . .                                                 | 32 |
| 6.1 | Detection accuracy comparison between distributed and centralized frameworks. . . . . | 40 |
| 6.2 | Detection time breakdown of distributed and centralized frameworks. . . . .           | 42 |
| 6.3 | Lifecycle reconstruction contribution of each AI agent. . . . .                       | 43 |
| 6.4 | Autonomous and support-dependent detection behavior of AI agents. . . . .             | 44 |
| 6.5 | Supporting relationship among AI agents during lifecycle reconstruction. . . . .      | 45 |
| 6.6 | Learning curve and mean lifecycle stop time. . . . .                                  | 47 |
| 6.7 | Distribution of selected response behaviors in the distributed framework . . . . .    | 49 |



# List of Tables

|     |                                                                                |    |
|-----|--------------------------------------------------------------------------------|----|
| 2.1 | Comparison of related works on AI-driven cyber threat detection and response . | 10 |
| 3.1 | Notations . . . . .                                                            | 13 |
| 4.1 | Response action space . . . . .                                                | 28 |
| 5.1 | List of tools and libraries used in the implementation . . . . .               | 33 |
| 5.2 | Physical host configuration . . . . .                                          | 33 |
| 5.3 | Virtualized testbed configuration for data collection . . . . .                | 34 |
| 5.4 | Docker deployment environment . . . . .                                        | 36 |
| 6.1 | TextCNN detection model configuration . . . . .                                | 38 |
| 6.2 | Reinforcement learning response model configuration . . . . .                  | 39 |
| 6.3 | Mean detection time under different window sizes. . . . .                      | 41 |
| 6.4 | Average selected step of response actions. . . . .                             | 48 |

# Chapter 1

## Introduction

Modern cyberattacks are increasingly multi-stage and often leave evidence across endpoints, networks, user accounts, and external connections. To defend against these threats, enterprise environments deploy multiple security solutions, such as Endpoint Detection and Response (EDR), Network Detection and Response (NDR), Intrusion Detection System (IDS), and firewalls. Each security solution monitors a specific part of the environment and generates alerts from its own perspective. However, these security solutions usually operate independently, causing alerts to lack shared context. As a result, one attack lifecycle may appear as many isolated alerts rather than a coherent attack progression.

In practice, these heterogeneous alerts are usually managed by a centralized Security Operations Center (SOC). The SOC commonly uses Security Information and Event Management (SIEM) systems to collect logs, aggregate alerts, and support event correlation. Security Orchestration, Automation and Response (SOAR) platforms can further help execute predefined workflows across security solutions. In this centralized process, alerts are collected into a central place, analyzed by analysts or supporting systems, and then response decisions are issued through security solutions. These responses may include blocking a malicious IP through a firewall, isolating a compromised host through EDR, disabling an account, resetting credentials, or dropping suspicious traffic.

Although SIEM and SOAR improve visibility and workflow automation, many decisions still depend on centralized analysis and human confirmation. Modern enterprise environments generate a large number of alerts, including redundant and low-priority ones, which increases SOC workload and causes alert fatigue. In addition, the SOC often waits for alerts from multiple sources to obtain a broader view before deciding the response. This centralized collection and correlation process delays mitigation, especially when attacks progress quickly across multiple

stages.

To reduce SOC workload, prior studies have applied AI-based methods to improve centralized alert analysis, intrusion detection, and response decision-making. Some works focus on alert classification, anomaly detection, and prioritization in SOC or SIEM-based environments [1], [2], [3]. Other studies further introduce AI-driven automation and reinforcement learning to support security operation and response optimization [4], [5]. Although these approaches improve the efficiency of security analysis and response, they are still mainly designed around centralized security operation, where alerts and decisions are collected, analyzed, and coordinated through a central SOC or SIEM-oriented architecture.

Recent cybersecurity trends suggest a shift from purely centralized security operation toward more distributed and composable security architectures. Gartner’s 2024 cybersecurity trends emphasize improving cybersecurity resilience under increasingly complex environments [6], while Cybersecurity Mesh Architecture highlights the need to extend security controls across distributed assets [7]. Following this direction, decentralized and multi-agent approaches have been explored for zero-trust architecture and distributed intrusion detection [8], [9]. However, these studies still mainly focus on specific security functions, such as trust management or intrusion detection, rather than heterogeneous enterprise security solutions working together. Therefore, limited research has jointly considered distributed AI agents across EDR, NDR, IDS/IPS, and firewall systems for lifecycle-level threat detection and coordinated response.

This contrast can be illustrated by the analogy of “Big Mom versus Smart Kids.” In a traditional SOC-based model, the central SOC acts as a “Big Mom” that collects all alerts, analyzes the situation, and decides what each security solution should do. This model provides a broad view but introduces delay and central dependency. In contrast, the proposed “Smart Kids” model embeds one AI Agent into each security solution. Each agent analyzes local evidence, communicates with peer agents when additional context is needed, and participates in response decision-making. This enables the system to respond closer to where evidence is generated instead of waiting for one central component to understand everything.

To address these challenges, this research proposes a distributed AI Agent-driven framework for coordinated threat detection and response across EDR, NDR, IDS, and firewall. The framework contains three main modules. The Communication Module uses a leader mechanism to exchange relevant evidence among agents while avoiding duplicated processing. The Detection Module uses a lightweight TextCNN model to classify structured and semi-structured alerts into MITRE ATT&CK techniques [10]. The Response Module uses reinforcement learning to select response actions based on the reconstructed lifecycle and current system state. Together, these modules support distributed detection, lifecycle reconstruction, and coordinated response without requiring all raw alerts to be centralized first.

Our work makes three key contributions to distributed AI-based threat detection and response in enterprise security environments:

1. We address the limitation of SOC-centered architectures, where heterogeneous alerts are centrally collected and manually or semi-automatically correlated, causing delayed response.
2. We propose an integrated distributed AI Agent framework that combines leader-based communication, TextCNN-based technique detection, lifecycle reconstruction, and reinforcement learning-based response selection.
3. We evaluate the framework in a controlled enterprise-like testbed, showing that distributed agents can achieve near-centralized detection accuracy while reducing detection time and improving response latency.

The remainder of this thesis is organized as follows. Chapter 2 presents the background and related works. Chapter 3 defines the problem statement. Chapter 4 describes the proposed methodology and system architecture. Chapter 5 details the implementation and experimental testbed. Chapter 6 presents experimental results and evaluation. Finally, Chapter 7 concludes the thesis and discusses future research directions.

# Chapter 2

## Background and Related Works

This chapter presents the foundational background and related works necessary for understanding this research. It introduces key cybersecurity components, attack modeling frameworks, and AI techniques used in detection and response, while also highlighting the limitations of existing approaches. This provides the basis for motivating a more distributed and collaborative security framework in the following chapters

### 2.1 Security Solutions

Modern enterprise cybersecurity relies on multiple specialized security solutions, each designed to monitor and protect a specific attack surface within the system. Instead of using a single defense mechanism, organizations commonly deploy a combination of endpoint, network, intrusion detection, and traffic control solutions to provide layered protection across the enterprise environment.

Endpoint Detection and Response (EDR) focuses on monitoring host-level activities, such as process execution, file access, registry changes, and abnormal system behaviors. It is commonly used to detect malware, suspicious processes, and endpoint compromise. Representative EDR or endpoint security platforms include Microsoft Defender for Endpoint [11], CrowdStrike Falcon [12], and Wazuh [13].

Network Detection and Response (NDR) analyzes network traffic to identify suspicious communication patterns, such as command-and-control traffic, lateral movement, and abnormal data transfer. Representative NDR or network security monitoring solutions include Darktrace [14], Vectra AI [15], and Zeek [16].

Intrusion Detection Systems (IDS) inspect network packets or traffic flows to detect known attack patterns based on rules, signatures, or behavioral indicators. IDS solutions are often deployed to identify scanning, exploitation attempts, and malicious network payloads. Representative open-source IDS/IPS tools include Snort [17] and Suricata [18].

Firewalls enforce network security policies by controlling inbound and outbound traffic based on addresses, ports, protocols, and predefined rules. They are commonly used to block unauthorized access, segment networks, and restrict malicious connections. Representative firewall solutions include Fortinet FortiGate [19], pfSense [20], and OPNsense [21].

## **2.2 Security Operation Center (SOC)**

In modern enterprise environments, a large number of security events are continuously generated from various security solutions such as EDR, NDR, IDS, and firewalls. To manage and analyze this data, organizations rely on the Security Operation Center (SOC), which serves as the central unit for cybersecurity operations. The SOC is responsible for monitoring security events, collecting alerts from multiple sources, analyzing potential incidents, and coordinating response activities. Through these functions, the SOC provides a centralized view of the security status of an organization.

A typical SOC workflow includes several major stages. First, security data is collected from different sources, including endpoint logs, network traffic, intrusion alerts, authentication records, and firewall events. Next, the collected events are monitored and analyzed to identify suspicious activities or potential incidents. When an incident is confirmed, SOC analysts investigate the related evidence, determine the affected assets, assess the severity of the incident, and coordinate suitable response actions. After the response process, reports are often generated to support documentation, compliance, and future security improvement.

Several technologies are commonly used to support SOC operations. Security Information and Event Management (SIEM) systems provide centralized log collection, event normalization,

correlation rules, dashboards, and reporting functions. Representative SIEM or SIEM-related platforms include Splunk Enterprise Security [22] and Wazuh [13].

Security Orchestration, Automation, and Response (SOAR) platforms further support SOC operations by connecting different security tools and automating response workflows. SOAR systems can execute predefined playbooks, enrich alerts with threat intelligence, create incident tickets, and trigger response actions such as blocking an IP address or isolating a host. Representative SOAR platforms include Palo Alto Cortex XSOAR [23] and Shuffle [24]. In addition, AI-based techniques have been increasingly integrated into SOC environments to support anomaly detection, alert classification, threat intelligence analysis, and incident prioritization.

## **2.3 MITRE ATT&CK and Attack Chain Representation**

The MITRE ATT&CK framework [10] is a widely used knowledge base that systematically describes adversarial behaviors through tactics and techniques. Tactics represent the high-level objectives of an attacker, such as Initial Access, Execution, Privilege Escalation, Lateral Movement, Exfiltration, and Impact. Techniques describe the concrete actions used to achieve these objectives. In enterprise environments, ATT&CK provides a structured representation of attacker behaviors across different attack stages, as shown in Figure 2.1.

Cyberattacks can be represented as multi-stage processes, where attackers execute a sequence of techniques to reach their final objectives. This sequence is often referred to as an attack lifecycle or attack chain. By mapping observed behaviors to ATT&CK techniques, security analysis can describe attack progression in a standardized form. Such representation is useful for organizing alerts, comparing attack scenarios, and describing how different techniques are connected across tactics.

| Reconnaissance<br>12 techniques        | Resource Development<br>9 techniques | Initial Access<br>11 techniques         | Execution<br>20 techniques            | Persistence<br>22 techniques             | Privilege Escalation<br>13 techniques    | Stealth<br>30 techniques                    | Defense Evasion<br>18 techniques                | Credential Access<br>17 techniques     | Discovery<br>34 techniques           | Lateral Movement<br>9 techniques        | Collection<br>17 techniques            | Command and Control<br>18 techniques      | Exfiltration<br>9 techniques               | Impact<br>15 techniques        |
|----------------------------------------|--------------------------------------|-----------------------------------------|---------------------------------------|------------------------------------------|------------------------------------------|---------------------------------------------|-------------------------------------------------|----------------------------------------|--------------------------------------|-----------------------------------------|----------------------------------------|-------------------------------------------|--------------------------------------------|--------------------------------|
| Active Scanning (2)                    | Acquire Access (1)                   | Content Injection (1)                   | BITS Jobs (1)                         | Account Manipulation (7)                 | Abuse Elevation Control Mechanism (6)    | Access Token Manipulation (6)               | Disable or Modify System Firewall (2)           | Adversary-in-the-Middle (4)            | Account Discovery (4)                | Exploitation of Remote Services (1)     | Adversary-in-the-Middle (4)            | Application Layer Protocol (2)            | Automated Exfiltration (1)                 | Account Access Removal (1)     |
| Gather Victim Host Information (4)     | Acquire Infrastructure (1)           | Drive-by Compromise (1)                 | Cloud Administration Command (1)      | BITS Jobs (1)                            | Access Token Manipulation (4)            | Build Image on Host (1)                     | Disable or Modify Tools (4)                     | Brute Force (4)                        | Application Window Discovery (1)     | Internal Spearfishing (1)               | Archive Collected Data (2)             | Communication Through Removable Media (1) | Data Transfer Size Limits (1)              | Data Destruction (1)           |
| Gather Victim Identity Information (2) | Compromise Accounts (2)              | Exploit Public-Facing Application (1)   | Command and Scripting Interpreter (1) | Boot or Logon Autostart Execution (14)   | Account Manipulation (7)                 | Debugger Evasion (1)                        | Domain or Tenant Policy Modification (2)        | Credentials from Password Stores (8)   | Browser Information Discovery (1)    | Lateral Tool Transfer (1)               | Audio Capture (1)                      | Content Injection (1)                     | Exfiltration Over Alternative Protocol (2) | Data Encrypted for Impact (1)  |
| Gather Victim Network Information (4)  | Compromise Infrastructure (2)        | External Remote Services (1)            | Container Administration Command (1)  | Boot or Logon Initialization Scripts (3) | Boot or Logon Autostart Execution (14)   | Deobfuscate/Decode Files or Information (1) | Downgrade Attack (1)                            | Exploitation for Credential Access (1) | Cloud Infrastructure Discovery (1)   | Cloud Service Dashboard (1)             | Remote Service Session Hijacking (2)   | Automated Collection (1)                  | Exfiltration Over C2 Channel (1)           | Data Manipulation (2)          |
| Gather Victim Org Information (4)      | Develop Capabilities (4)             | Hardware Additions (1)                  | Deploy Container (1)                  | Cloud Application Integration (1)        | Boot or Logon Initialization Scripts (3) | Direct Volume Access (1)                    | Exploitation for Defense Impairment (1)         | Forced Authentication (1)              | Cloud Service Object Discovery (1)   | Cloud Storage Discovery (1)             | Remote Services (2)                    | Clipboard Data (1)                        | Data Encoding (2)                          | Defacement (2)                 |
| Phishing for Information (4)           | Establish Accounts (2)               | Phishing (4)                            | ESXi Administration Command (1)       | Compromise Host Software Binary (1)      | Boot or Logon Initialization Scripts (3) | Execution Guardrails (2)                    | File and Directory Permissions Modification (2) | Forge Web Credentials (2)              | Container and Resource Discovery (1) | Replication Through Removable Media (1) | Remote Session Hijacking (1)           | Data from Cloud Storage (1)               | Data Obfuscation (2)                       | Disk Wipe (2)                  |
| Query Public AI Services (1)           | Generate Content (2)                 | Replication Through Removable Media (1) | Exploitation for Client Execution (1) | Create Account (2)                       | Create or Modify System Process (2)      | Hide Artifacts (14)                         | Modify Authentication Process (2)               | Input Capture (4)                      | Device Driver Discovery (1)          | Software Deployment Tools (1)           | Clipboard Data (1)                     | Data from Configuration Repository (1)    | Dynamic Resolution (2)                     | Email Bombing (1)              |
| Search Closed Sources (2)              | Obtain Capabilities (7)              | Supply Chain Compromise (2)             | Hijack Execution Flow (12)            | Create or Modify System Process (2)      | Domain or Tenant Policy Modification (2) | Hijack Execution Flow (12)                  | Modify Authentication Process (2)               | Multi-Factor Authentication (1)        | Debugger Evasion (1)                 | Software Deployment Tools (1)           | Clipboard Data (1)                     | Data from Configuration Repository (1)    | Encrypted Channel (2)                      | Endpoint Denial of Service (4) |
| Search Open Technical Databases (6)    | Stage Capabilities (4)               | Trusted Relationship (1)                | Input Injection (1)                   | Event Triggered Execution (12)           | Escape to Host (1)                       | Indicator Removal (2)                       | Modify Cloud Infrastructure (1)                 | Multi-Factor Authentication (1)        | Domain Trust Discovery (1)           | Taint Shared Content (1)                | Data from Information Repositories (1) | Failback Channels (1)                     | Exfiltration Over Physical Medium (1)      | Financial Theft (1)            |
| Search Open Websites/Domains (2)       | Valid Accounts (4)                   | Inter-Process Communication (2)         | Event Triggered Execution (12)        | Event Triggered Execution (12)           | Event Triggered Execution (12)           | Indirect Command Execution (1)              |                                                 |                                        | File and Directory Discovery (1)     | Hide Infrastructure (1)                 | Data from Local (1)                    | Infrastructure (1)                        | Exfiltration Over Web Service (4)          | Firmware Corruption (1)        |
|                                        |                                      |                                         |                                       |                                          |                                          |                                             |                                                 |                                        |                                      |                                         |                                        |                                           | Scheduled Transfer (1)                     | Inhibit System Recovery (1)    |

Figure 2.1: MITRE ATT&CK matrix in enterprise network

A ransomware attack can be illustrated using the ATT&CK framework as a sequence of stages. The attack may begin with Initial Access (T1566—Phishing), where a malicious email delivers the payload. This is followed by Execution (T1059—Command and Scripting Interpreter) to run malicious code, and Persistence (T1547—Boot or Logon Autostart Execution) to maintain access on the system. The attacker may then perform Privilege Escalation (T1068—Exploitation for Privilege Escalation) and Credential Access (T1003—OS Credential Dumping) to obtain higher privileges and extract credentials. After that, Lateral Movement (T1021—Remote Services) can be used to spread across the network. Finally, the attack reaches the Impact stage (T1486—Data Encrypted for Impact), where files are encrypted and a ransom is demanded.

This example shows how ATT&CK can describe a cyberattack as an ordered chain of techniques rather than a single isolated event. With this representation, attack behaviors can be organized according to their tactical purposes and execution order, providing a clear structure for discussing attack progression in later chapters.

## 2.4 AI-Driven Detection and Response in Cybersecurity

Artificial intelligence has been increasingly applied in cybersecurity to support threat detection and response. Compared with traditional rule-based systems, AI-driven approaches can learn patterns from historical data and assist in identifying suspicious behaviors from large volumes of security events. In this context, AI techniques are commonly used for tasks such as alert classification, anomaly detection, attack technique identification, and response decision-making.



At the system level, AI agents are commonly used to represent autonomous components that can observe their environment, analyze security events, and perform actions. Each agent typically consists of three main functionalities: perception—collecting alerts and logs, reasoning—analyzing potential threats, and action—executing response strategies. In cybersecurity systems, AI agents may be deployed to monitor local security data, detect malicious activities, exchange information with other components, and support response decisions.

Security data is often represented as textual or semi-structured information, including system logs, network alerts, firewall records, command lines, file paths, process names, and event descriptions. These data sources contain meaningful patterns that can indicate malicious behaviors, such as suspicious commands, abnormal connections, or known attack signatures. Prior studies have also modeled system logs as sequential textual data for anomaly detection, showing that log patterns can be learned by deep learning models [25]. Therefore, text-based machine learning and deep learning methods are suitable for processing security alerts, since they can transform raw textual events into numerical representations and learn discriminative patterns for detection tasks.

Several AI models have been applied to text-based cybersecurity detection. Traditional machine learning models such as Support Vector Machine (SVM) are often used for alert classification when features are manually extracted from logs [26]. More advanced deep learning models, such as Convolutional Neural Networks (CNNs), can automatically capture local patterns from token sequences, making them useful for detecting important keywords, command fragments, or short event patterns [27]. Transformer-based models further improve contextual representation by using self-attention mechanisms to capture relationships among tokens across longer sequences [28, 29]. Recently, Large Language Models (LLMs) have also been explored for cybersecurity analysis because of their ability to process complex textual descriptions and support reasoning-based security tasks [30].

Although large models such as Transformers and LLMs provide strong representation capability, they usually require higher computational resources and longer inference time. In dis-

tributed security architectures, where AI agents are expected to be embedded near individual security solutions, lightweight models can be more practical. TextCNN is one such model that applies convolutional filters over token embeddings to extract local textual features. Since security alerts are often short, structured, and keyword-rich, TextCNN can capture useful detection patterns with lower computational cost than more complex architectures. This makes it suitable for resource-constrained AI agents deployed inside or near security tools such as EDR, NDR, IDS, and firewalls.

In addition to detection, cyber defense also involves selecting appropriate response actions based on the current state of an attack. This can be formulated as a sequential decision-making problem, where the system determines which action to take after observing the current situation. Reinforcement Learning (RL), as illustrated in Figure 2.2, is a learning paradigm where an agent interacts with an environment by taking actions and receiving feedback in the form of rewards, with the objective of learning a policy that maximizes cumulative reward over time [31]. In cybersecurity, RL can be used to model response selection, where different actions may have different effects and operational costs [4].

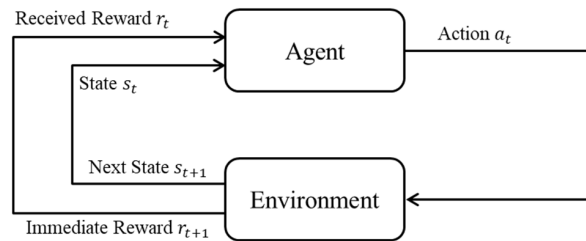


Figure 2.2: The agent-environment interaction in RL

## 2.5 Related Works on AI-Driven and Decentralized Threat Detection and Response

Table 2.1 compares representative studies on cybersecurity detection and response from four perspectives. First, it distinguishes the architectural design, including centralized systems without AI agents, centralized systems with AI-agent-based automation, decentralized approaches, and the proposed distributed AI-agent framework. Second, it compares the target environment

of each work, such as enterprise networks, wireless/6G networks, and large-scale networks. Third, it examines the automated tasks, including the detected object and whether automated response is supported. Finally, it summarizes the covered security components and methodologies, including SIEM/SOAR, IDS, firewall, IPS, EDR, NDR, machine learning, reinforcement learning, blockchain, federated learning, and multi-agent techniques.

Table 2.1: Comparison of related works on AI-driven cyber threat detection and response

| Work        | Category                     | Target Env.           | Automated Tasks            |             | Security Components |     |          |     |     |     | Methodology                   |
|-------------|------------------------------|-----------------------|----------------------------|-------------|---------------------|-----|----------|-----|-----|-----|-------------------------------|
|             |                              |                       | Detected Object            | Auto. Resp. | SIEM/SOAR           | IDS | Firewall | IPS | EDR | NDR |                               |
| [1]         | Centralized without AI Agent | Enterprise network    | Security alert triage      | –           | O                   | –   | –        | –   | –   | –   | Imbalanced ML                 |
| [2]         |                              | Enterprise network    | Intrusion alerts           | –           | O                   | O   | –        | –   | –   | –   | ML-based SIEM-IDS integration |
| [32]        |                              | Network traffic       | Abnormal traffic           | –           | –                   | O   | –        | –   | –   | O   | Semi-supervised DRL           |
| [3]         | Centralized with AI Agent    | Enterprise network    | Security incidents         | O           | O                   | –   | –        | –   | –   | –   | AI-assisted SIEM / SOAR       |
| [33]        |                              | Enterprise network    | Security incidents         | O           | O                   | –   | –        | –   | –   | –   | AI-based SOAR                 |
| [34]        |                              | Enterprise network    | Botnet, DDoS, malware      | O           | O                   | O   | –        | –   | –   | –   | SOAR-based dynamic honeypots  |
| [4]         |                              | Enterprise network    | Multi-stage attacks        | O           | –                   | –   | –        | –   | –   | –   | RL                            |
| [5]         |                              | Enterprise network    | Security workflows         | O           | O                   | –   | –        | –   | –   | –   | Agentic LLM-based SOAR        |
| [8]         | Decentralized                | Wireless / 6G network | Trust and access anomalies | O           | –                   | –   | –        | –   | –   | –   | Blockchain + FL + ZTA         |
| [9]         |                              | Large-scale network   | Intrusions                 | –           | –                   | O   | –        | –   | –   | O   | Decentralized MARL IDS        |
| <b>Ours</b> | <b>Distributed AI Agents</b> | Enterprise network    | Attack lifecycles          | O           | –                   | O   | O        | O   | O   | O   | Coordinated TextCNN-RL agents |

DRL = Deep Reinforcement Learning; MARL = Multi-Agent Reinforcement Learning; FL = Federated Learning; ZTA = Zero Trust Architecture; IRS = Intrusion Response System.

Existing cybersecurity automation studies first focus mainly on centralized detection and alert analysis. For example, [1] addresses security alert analysis under imbalanced data conditions, while [2] integrates SIEM and IDS data with machine learning for live intrusion analysis. Similarly, [32] applies semi-supervised deep reinforcement learning to detect abnormal network traffic. These works improve the quality of alert classification and intrusion detection in enterprise or network traffic settings. However, they mainly focus on detection or alert analysis, while automated response and coordination across heterogeneous security tools are not the main objectives.

More recent studies introduce AI-driven automation into centralized security operations. Works such as [3], [33], and [34] improve SIEM/SOAR-based security workflows by supporting alert prioritization, orchestration, or automated response. Reinforcement learning has also been applied to intrusion response systems, as shown in [4], where the response policy is optimized for advanced multi-stage threats. In addition, [5] explores agentic AI for hyper-automation in security operation centers. Although these methods improve automation and decision-making, they still rely on centralized coordination, where alerts, context, and response decisions are collected or orchestrated by a central system. This design may introduce response delay, scalability limitations, and a potential single point of failure in large enterprise networks.

To reduce the dependency on centralized security operation, several studies investigate decentralized cybersecurity designs. For instance, [8] studies decentralized zero-trust architecture for wireless and 6G networks using blockchain and federated learning. Another work, such as [9], applies multi-agent reinforcement learning to distributed intrusion detection. These studies show that decentralized and multi-agent designs can improve scalability and reduce centralized dependency. However, they are mostly focused on IDS/NDR-oriented detection, decentralized trust management, or specific network settings. They do not jointly address lifecycle-level threat understanding and automated response across heterogeneous enterprise security components.

In contrast, this research proposes a distributed AI-agent-driven framework for coordinated threat detection and response in enterprise networks. Each security component, including EDR, NDR, IDS/IPS, and firewall, is equipped with a dedicated AI agent that performs local detection, context sharing, and response decision-making. Instead of forwarding all raw data to a centralized analysis point, agents exchange relevant evidence and collaboratively reconstruct the attack lifecycle. Based on the reconstructed lifecycle and current system state, the response module selects defensive actions by considering both mitigation effectiveness and operational impact. Compared with prior works, the proposed framework uniquely combines distributed AI agents, heterogeneous security-tool coverage, lifecycle-level reasoning, and reinforcement-learning-based response optimization.

# Chapter 3

## Problem Formulation

This chapter formulates the problem of coordinated threat detection and response in a distributed AI-agent-based cybersecurity framework. The chapter is organized into two parts: (1) the notation table, which defines key symbols and variables used throughout the formulation, and (2) the problem statement, which formalizes the objectives of technique detection, lifecycle correlation, and response action selection under a distributed setting.

### 3.1 Notations

To describe the distributed detection, communication, correlation, and response process, we adopt a unified set of notations summarized in Table 3.1. These notations are grouped into four main categories: (1) AI Agents, which represent the distributed entities attached to different security tools, (2) Communication, which describe how agents exchange contextual information, (3) Detection, which defines how collected data are transformed into MITRE ATT&CK techniques, and (4) Response, which represents the reinforcement learning-based action selection process.

In the AI Agents category,  $N_A$  denotes the number of AI Agents in the system, and  $A = \{A_i \mid i = 1, 2, \dots, N_A\}$  denotes the set of all AI Agents. Each  $A_i$  represents one agent assigned to a specific security tool, such as EDR, NDR, IDS, or Firewall. The index  $i$  is used to identify each individual agent in the distributed framework. Each agent is responsible for collecting local data, performing detection, communicating with peer agents, and participating in the response process.

In the Communication category,  $E_{t,i}$  denotes the local data collected by Agent  $A_i$  at step  $t$ . This data is obtained from the security tool assigned to the agent and may contain logs, alerts, or events observed within the current time step. The notation  $H$  denotes host-related information extracted from the given input, such as host name, source IP, destination IP, or

Table 3.1: Notations

| Category      | Symbol                                              | Term                                                                                                                                |
|---------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| AI Agents     | $N_A$                                               | Number of AI Agents in the system                                                                                                   |
|               | $i$                                                 | Index of AI Agents                                                                                                                  |
|               | $A = \{A_i \mid i = 1, 2, \dots, N_A\}$             | Set of AI Agents                                                                                                                    |
| Communication | $E_{t,i}$                                           | Local data collected by Agent $A_i$ at step $t$                                                                                     |
|               | $H$                                                 | Information about the hosts related to the given input $E_{t,i}$ , such as host name, source IP, or destination IP                  |
|               | $Req_i = \{q_{i,k} \mid k = 1, 2, \dots,  Req_i \}$ | Request list of Agent $A_i$                                                                                                         |
|               | $q_{i,k}$                                           | Request from another Agent to Agent $A_i$                                                                                           |
|               | $q_{i,k}.requester$                                 | The Agent that sends request $q_{i,k}$ to Agent $A_i$ , where $q_{i,k}.requester \in A \setminus \{A_i\}$                           |
|               | $q_{i,k}.hosts$                                     | Host information in request $q_{i,k}$                                                                                               |
| Detection     | $N_M$                                               | Number of MITRE ATT&CK techniques available in CTI                                                                                  |
|               | $M = \{m_1, m_2, \dots, m_{N_M}\}$                  | Set of predefined MITRE ATT&CK techniques                                                                                           |
|               | $E_t$                                               | Input data for the detection model at step $t$ , made up from local data $E_{t,1}, E_{t,2}, \dots, E_{t,N_A}$ through communication |
|               | $T_t$                                               | Detected technique at step $t$ , where $T_t \in M$                                                                                  |
| Response      | $L_t = \{T_1, T_2, \dots, T_t\}$                    | Reconstructed lifecycle as a list of techniques after communication at step $t$                                                     |
|               | $Sys_t$                                             | System state at step $t$                                                                                                            |
|               | $S_t$                                               | State of the RL module, which is a combination of $L_t$ and $Sys_t$                                                                 |
|               | $N_U$                                               | Number of available response actions of the response module                                                                         |
|               | $U = \{u_1, u_2, \dots, u_{N_U}\}$                  | Set of available response actions of the response module                                                                            |
|               | $u_t^* \in U$                                       | Selected response action at step $t$                                                                                                |
|               | $R_t$                                               | Reward of action $u_t^*$                                                                                                            |

other entity-related information. This information is used as contextual evidence for matching relevant data across different agents. Each agent maintains a request list denoted by  $Req_i = \{q_{i,k} \mid k = 1, 2, \dots, |Req_i|\}$ . Each  $q_{i,k}$  represents a request received by Agent  $A_i$  from another agent. The sender of the request is denoted by  $q_{i,k}.requester$ , where  $q_{i,k}.requester \in A \setminus \{A_i\}$ . The host-related information carried in the request is denoted by  $q_{i,k}.hosts$ . Through this request mechanism, agents can ask peer agents for additional evidence related to the same host, IP address, or suspicious entity. This allows the framework to enrich local observations with distributed contextual information before finalizing the detection result.

In the Detection category,  $N_M$  denotes the number of MITRE ATT&CK techniques available in the cyber threat intelligence knowledge base, and  $M = \{m_1, m_2, \dots, m_{N_M}\}$  denotes the set of predefined MITRE ATT&CK techniques. At step  $t$ ,  $E_t$  represents the input data for the detection

model. This input is constructed from local data and additional information obtained through communication among agents. Based on  $E_t$ , the detection model outputs  $T_t$ , which denotes the detected MITRE ATT&CK technique at step  $t$ , where  $T_t \in M$ .

In the Response category,  $L_t$  denotes the reconstructed attack lifecycle after communication and detection at step  $t$ . It is represented as a list of detected MITRE ATT&CK techniques accumulated over time. The notation  $Sys_t$  denotes the system state at step  $t$ , which includes response-related information such as whether a malicious IP, compromised host, compromised account, suspicious process, or malicious flow is known. The reinforcement learning state is denoted by  $S_t$ , which combines the reconstructed lifecycle  $L_t$  and the system state  $Sys_t$ .

Finally,  $N_U$  denotes the number of available response actions, and  $U = \{u_1, u_2, \dots, u_{N_U}\}$  denotes the set of available actions in the response module. At step  $t$ , the selected response action is denoted by  $u_t^* \in U$ . After executing this action, the response module receives a reward denoted by  $R_t$ . Together, these notations provide a consistent representation of how the proposed framework performs distributed data collection, agent communication, technique detection, lifecycle reconstruction, and response action selection.

## 3.2 Problem Statement

In a real internal network, different security solutions observe different parts of the same attack. For example, an EDR may observe suspicious processes on a host, an IDS may detect abnormal packets, an NDR may observe suspicious network flows, and a Firewall may record blocked or allowed connections. However, each solution only sees a partial view of the attack. If these observations are analyzed independently, the system may fail to understand how the attack is progressing. Therefore, this research considers a distributed defense scenario where each security solution is equipped with an AI Agent. These agents do not simply work alone; instead, they cooperate by sharing useful context and jointly supporting detection and response. The overall goal is to detect the current attack technique from distributed evidence and then select an appropriate response action based on the detected lifecycle and the current system condition.

Formally, as illustrated in Figure 3.1, the system contains  $N_A$  security solutions in the internal network, and each solution is associated with one AI Agent. The set of AI Agents is denoted by  $A = \{A_i \mid i = 1, 2, \dots, N_A\}$ . At step  $t$ , each Agent  $A_i$  collects local data  $E_{t,i}$  from its assigned security solution. Through communication among agents, the local observations are combined into the detection input  $E_t$ . Given  $E_t$  and the predefined MITRE ATT&CK technique set  $M$ , the detection module identifies the current detected technique  $T_t$ . After detection, the detected technique is used to update the reconstructed lifecycle  $L_t$ . The response module then takes  $L_t$ , the current system state  $Sys_t$ , and the available response action set  $U$  as input, and selects a response action  $u_t^*$  to stop or mitigate the attack. Therefore, the problem is formulated as two connected subproblems: attack progression detection and response action selection.

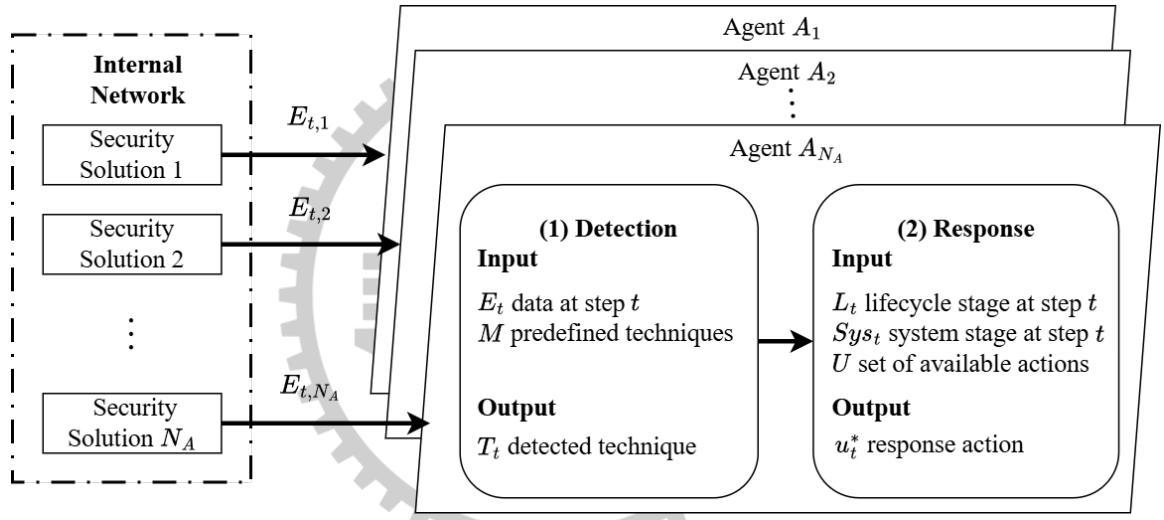


Figure 3.1: Detection and response with distributed AI agents

### 1) Attack Progression Detection

The first subproblem is attack progression detection. The purpose of this subproblem is to identify the MITRE ATT&CK technique that represents the current attack behavior at step  $t$ . Since each AI Agent only observes data from its assigned security solution, the detection input may be constructed not only from local data but also from additional context obtained through communication with peer agents.

#### Input:

- Collection of alerts or local data from multiple agents through communication, denoted by  $E_t$ , which is made up from  $E_{t,i}$  collected by agents  $A_i \in A$



- Set of predefined MITRE ATT&CK techniques  $M = \{m_1, m_2, \dots, m_{N_M}\}$

**Output:**

- Detected technique  $T_t$ , where  $T_t \in M$

**Objective:**

The objective is to maximize the detection accuracy of  $T_t$  compared with the ground-truth MITRE ATT&CK technique. In other words, the detection module should correctly map distributed security observations into the corresponding attack technique.

**Constraints:**

- The detection process is performed by  $N_A$  distributed AI Agents
- Each AI Agent operates under low computing resource conditions

After  $T_t$  is detected, it is added to the reconstructed lifecycle  $L_t$ . In this way, attack progression detection does not only identify an isolated technique, but also continuously supports the construction of the current attack lifecycle.

## 2) Response Action Selection

The second subproblem is response action selection. After the attack progression is detected and the lifecycle is updated, the framework must decide which response action should be executed. The selected action should be effective enough to stop or mitigate the attack, while avoiding unnecessary operational cost or overly aggressive responses.

**Input:**

- Current reconstructed lifecycle  $L_t$  at step  $t$
- Current system state  $Sys_t$
- Set of available response actions  $U = \{u_1, u_2, \dots, u_{N_U}\}$

**Output:**

- Selected response action  $u_t^* \in U$

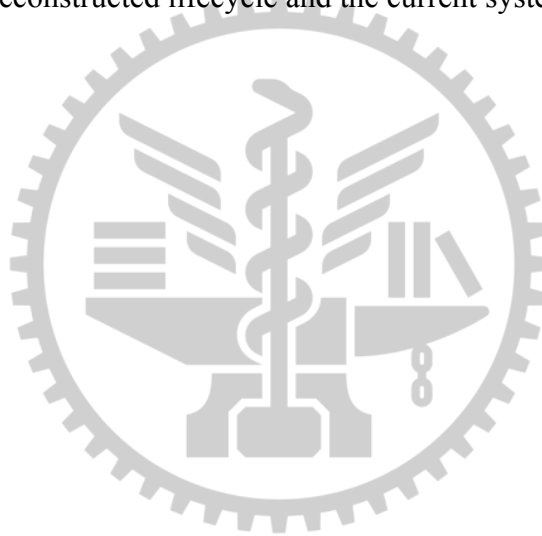
**Objective:**

The objective is to maximize the estimated successful blocking possibility of the detected lifecycle. The response module should select an action that can effectively stop the attack while considering the cost and impact of the action on the system.

**Constraints:**

- The number of available response actions is limited to  $N_U$

Together, these two subproblems form the core problem of this thesis. The first subproblem focuses on understanding what attack technique is currently happening by using distributed evidence from multiple agents. The second subproblem focuses on deciding how the system should respond based on the reconstructed lifecycle and the current system condition.



# Chapter 4

## Distributed AI Agent-Driven Framework for Detection and Response

This chapter presents the methodology of the proposed distributed AI agent-driven framework for coordinated threat detection and response, organized into two main sections. First, the overall flow of the solution is introduced to explain how the framework transforms local alerts into coordinated response decisions. Second, the internal architecture of each AI Agent is presented in detail, including the Communication Module, Detection Module, and Response Module.

### 4.1 Solution Overview

Based on the problem defined in the previous chapter, the main challenge is how to enable multiple security solutions to jointly detect attack progression and select an effective response action without relying on a fully centralized analysis component. Each security solution observes only a partial view of the attack. Therefore, the proposed framework assigns one AI Agent to each security solution and allows these agents to collaborate through communication. Instead of collecting all raw alerts into a central location, each agent processes its own local observations and exchanges only useful contextual information with peer agents when additional evidence is needed.

As shown in Figure 4.1, the internal network contains multiple security solutions, where each solution is associated with an AI Agent. The lower part of the figure shows the distributed deployment of agents across different security solutions, while the upper part zooms into the internal architecture of one agent. Each AI Agent is composed of three major modules: Communication, Detection, and Response. The Communication Module exchanges information with other agents through the dashed communication links. The collected and augmented input data  $E_t$  is

then passed to the Detection Module, which predicts the current MITRE ATT&CK technique  $T_t$ . After that, the detected technique is used by the Response Module to select an appropriate response action.

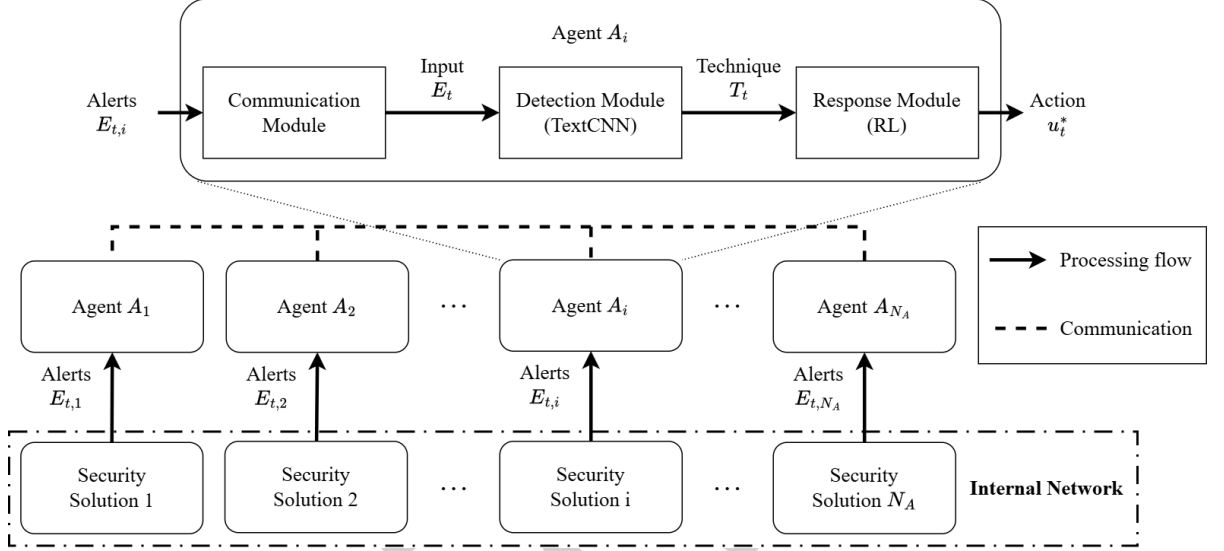


Figure 4.1: Overall framework and AI Agent architecture

The Communication Module is designed based on a collaboration mechanism with leader control. When an agent observes data related to an attack, it can communicate with peer agents to obtain additional context from other security perspectives. The leader mechanism is used to avoid duplicated processing and conflicting responses. Instead of allowing all agents to independently make response decisions for the same lifecycle, one agent coordinates the communication and continues the processing flow for that lifecycle. This design keeps the framework distributed while maintaining coordinated decision-making.

The Detection Module uses TextCNN to identify MITRE ATT&CK techniques from the input data  $E_t$ . This choice is suitable because security alerts are usually structured or semi-structured text, such as logs, rule messages, command traces, IP addresses, and event descriptions. These data often contain local textual patterns rather than complex natural language semantics. Therefore, a lightweight TextCNN model can capture useful alert patterns with lower computational cost than larger Transformer-based or LLM-based models. This makes it more suitable for distributed agents that need to perform fast local inference under limited computing resources.

The Response Module uses reinforcement learning to select the response action after the detected technique and lifecycle context are available. Since different response actions have different effectiveness and operational costs, the response problem is not only about stopping the attack, but also about avoiding unnecessarily aggressive actions. Reinforcement learning is adopted because it can learn action selection policies from interaction with the environment and balance attack-blocking effectiveness with response cost. Through this design, the agent can select actions according to the current attack progression and system state instead of applying fixed response rules.

Overall, the proposed solution follows a distributed but coordinated design. Communication with leader control enables agents to share useful evidence without centralizing all raw data. TextCNN provides lightweight and fast technique detection for each agent. Reinforcement learning supports adaptive response action selection under different attack and system conditions. Together, these three modules allow the framework to perform collaborative threat detection and response across heterogeneous security solutions.

## 4.2 Communication Module

The Communication Module is the backbone of the proposed distributed AI Agent framework. As discussed in the problem statement, each security solution only observes a partial view of an attack. Therefore, an agent should not immediately make an isolated decision whenever it receives new data. Instead, it first needs to determine whether its local observation should support an existing leader agent or whether it should start a new detection process by itself. This module is placed before the detailed Detection and Response Modules because it controls how agents collaborate, how duplicated processing is avoided, and how the framework maintains coordinated behavior in a distributed environment.

As shown in Figure 4.2, when Agent  $A_i$  collects new local data  $E_{t,i}$  at step  $t$ , it first checks its request list  $Req_i$ . Each request  $q_{i,k}$  in  $Req_i$  is sent by another agent that is already acting as a leader and waiting for additional evidence. The agent compares its new local data with the

request information, especially the host-related information  $q_{i,k}.hosts$ . This matching process can be based on information such as host name, source IP, destination IP, account, process, or flow information, depending on the security solution attached to the agent.

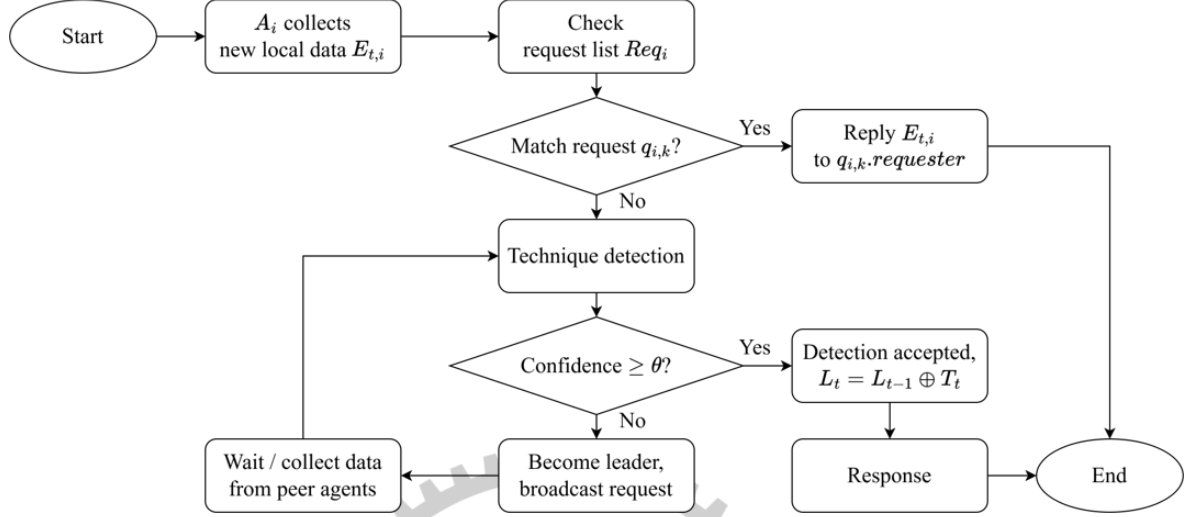


Figure 4.2: Communication flow with leader mechanism

A request is matched based on the host-related information contained in the local data and the request. Let  $H(E_{t,i})$  denote the host-related information extracted from the newly collected local data  $E_{t,i}$ , such as host name, source IP, destination IP, account, process, or flow information. Agent  $A_i$  checks each request  $q_{i,k} \in Req_i$  and compares  $H(E_{t,i})$  with  $q_{i,k}.hosts$ . If their intersection is not empty, the local data is considered relevant to that request:

$$H(E_{t,i}) \cap q_{i,k}.hosts \neq \emptyset.$$

In this case, Agent  $A_i$  does not start another independent processing flow. Instead, it replies with the relevant local data  $E_{t,i}$  to  $q_{i,k}.requester$ . Agent  $A_i$  then acts as a supporting agent for the existing leader, and the process ends for this local data. This design prevents multiple agents from repeatedly processing the same suspicious activity and avoids duplicated response decisions for the same attack behavior.

If no matched request exists in  $Req_i$ , Agent  $A_i$  continues to the Detection Module. The detection module analyzes the current input and estimates the detected technique. If the detection

confidence is greater than or equal to the threshold  $\theta$ , the detection result is accepted. The detected technique  $T_t$  is then added to the current lifecycle, represented as  $L_t = L_{t-1} \oplus T_t$ , and the process continues to the Response Module. In this case, the agent has enough confidence to proceed without asking peer agents for additional evidence.

If the detection confidence is lower than  $\theta$ , the local observation is considered insufficient. Agent  $A_i$  then becomes the leader for the current suspicious activity and broadcasts requests to peer agents. The purpose of this request is not to collect all raw data from all agents, but to ask whether other agents have relevant observations related to the same host or entity. The leader then waits for replies, augments its available evidence, and performs detection again. This loop continues until the detection result becomes confident enough or until the communication process reaches its waiting limit.

The waiting-time design is shown in Figure 4.3. After a leader broadcasts requests to peer agents, it does not wait indefinitely for every possible reply. Instead, replies are collected within a fixed waiting period. When this period ends, the leader continues the processing flow with the evidence that has already been received. This design is important for cyber defense because delaying the response too long may allow the attacker to move further in the attack lifecycle. Therefore, the module prioritizes timely decision-making while still allowing agents to enrich local evidence through collaboration.

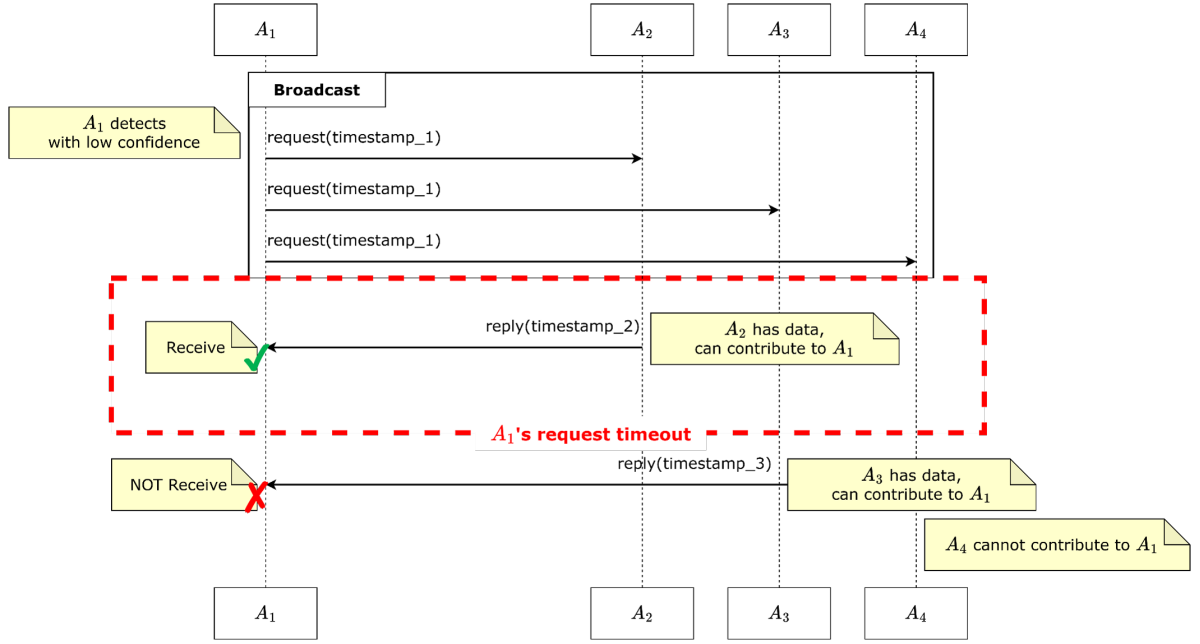


Figure 4.3: Response waiting time

Another important issue is leader role conflict, as illustrated in Figure 4.4. This situation occurs when more than one agent observes related suspicious data at nearly the same time and multiple agents attempt to become leaders. Without conflict handling, the framework may create repeated requests, duplicated detection flows, or inconsistent response actions. Therefore, the Communication Module includes a leader conflict handling mechanism to ensure that only one agent continues as the leader for the same suspicious activity. The other agents then act as supporting agents and reply with relevant local evidence when requested.

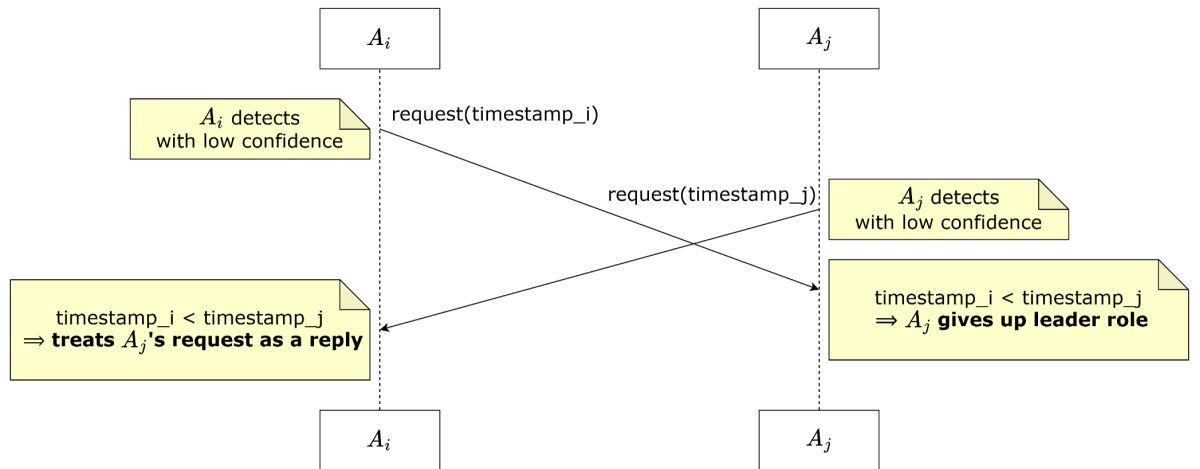


Figure 4.4: Leader role conflict handling



### 4.3 Detection Module

The Detection Module converts the input data obtained after communication into a MITRE ATT&CK technique. For each AI Agent  $A_i \in A$ , the input of this module is denoted by  $E_t$ , which contains the local data of the agent and, when needed, additional contextual data collected from peer agents through the Communication Module. The output is the detected technique  $T_t$ , where  $T_t \in M$ . As illustrated in Figure 4.5, the detection model is implemented using TextCNN. The input data is first transformed into a token embedding matrix. Then, multiple convolution filters with different kernel sizes are applied to capture local alert patterns. Max pooling is used to extract the most representative features from each convolution feature map. These features are concatenated into a single feature vector, passed through dropout and a fully connected layer, and finally converted into technique probabilities through the softmax output layer. The technique with the highest probability is selected as the detected technique  $T_t$ .

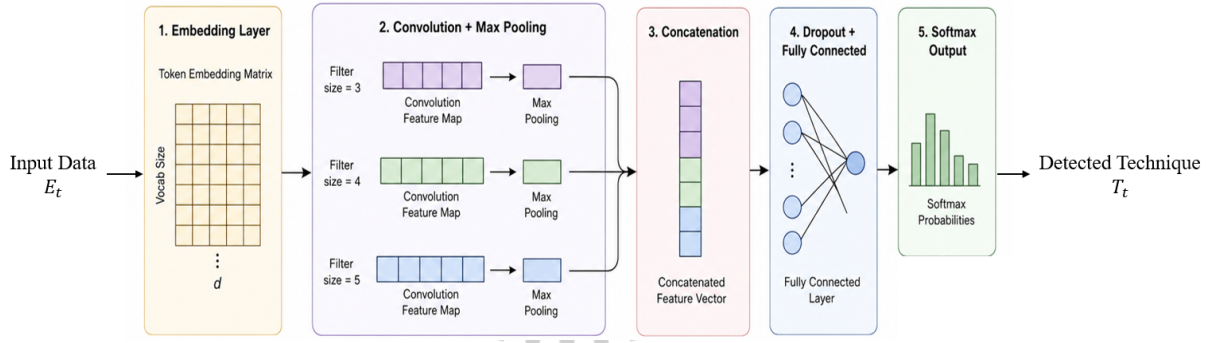


Figure 4.5: TextCNN-based Detection Module

TextCNN is selected because security alerts are usually structured or semi-structured textual data rather than complex natural language. These alerts often contain clear local patterns, such as commands, rule messages, IP addresses, ports, file paths, process names, and event descriptions. Therefore, the detection task does not necessarily require a complex Transformer or LLM-based architecture. By using convolution filters with different sizes, TextCNN can efficiently capture useful local patterns from alert text while keeping the model lightweight. This design reduces computational cost and inference time, making it suitable for distributed AI Agents that need to run detection locally under limited computing resources.

## 4.4 Response Module

The Response Module is responsible for selecting the response action after the current technique has been detected. As shown in Figure 4.6, the module receives the state  $S_t$ , which represents both the reconstructed attack lifecycle  $L_t$  and the current system state  $Sys_t$ . The lifecycle  $L_t$  describes how the attack has progressed up to step  $t$ , while  $Sys_t$  describes whether the system currently has enough information to execute specific response actions, such as a malicious IP address, compromised host, compromised account, suspicious process, or malicious flow. Based on  $S_t$ , the reinforcement learning model selects a response action  $u_t^*$  from the predefined action space  $U$ . After the selected action is executed, the environment returns a reward  $R_t$ , which is used to guide the learning process.

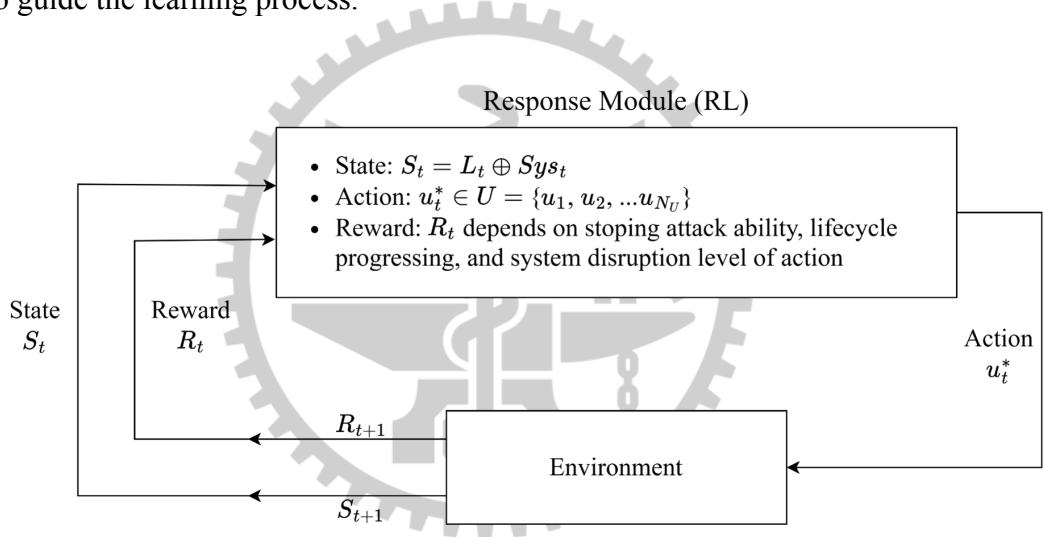


Figure 4.6: Reinforcement learning-based Response Module

The response process is modeled as a reinforcement learning problem because the best response action depends on both attack progression and system condition. A fixed rule-based response may be too rigid because different actions have different effectiveness and operational costs. For example, blocking a malicious IP is a low-impact action that can be useful at an early stage when the malicious source is known. In contrast, isolating a host or shutting down a machine can strongly stop the attack, but these actions are highly disruptive to normal operations. Therefore, the goal of the Response Module is not only to stop the attack, but also to select an action with an appropriate cost according to the current situation.

The state design of  $S_t$  follows two main ideas. First, it includes lifecycle-related information to describe the current attack progression. This part includes the detected lifecycle history, the current technique, the number of detected techniques, and the agent that detects or leads the current step. These features allow the RL model to understand whether the attack is still at an early stage or has already progressed deeper into the lifecycle. Second,  $S_t$  includes system-related information from  $Sys_t$  to describe which response actions are executable. For example, firewall blocking requires a known malicious IP, endpoint containment requires a compromised host, account-related actions require a compromised account, process termination requires a suspicious process, and NDR quarantine requires a malicious flow. By combining  $L_t$  and  $Sys_t$ , the RL model can make decisions based on both attack context and available response capability.

The reward design teaches the agent how to balance blocking effectiveness and response cost. A positive reward is given when the selected action successfully stops the lifecycle, and an additional preference is given to earlier stopping because earlier containment can prevent the attacker from reaching later stages. Low-impact actions are encouraged at earlier stages when they are sufficient to stop the attack. On the other hand, penalties are assigned for late response, high-cost actions, overly aggressive actions selected too early, and repeated actions that fail to stop the attack. This reward design guides the model to avoid unnecessary high-impact responses while still learning to select stronger actions when the attack has progressed further or when low-impact actions are insufficient.

Algorithm 4.1 summarizes the RL-based response action selection process. At each step, the detected technique  $T_t$  is first appended to the reconstructed lifecycle  $L_t$ . The Response Module then constructs the state  $S_t$  from  $L_t$  and  $Sys_t$ . Before selecting an action, the module checks the executable actions according to the current system state. The RL policy then selects one valid response action, executes it, observes the next system state, and receives a reward according to the result of the action.

---

**Algorithm 4.1** RL-based Response Action Selection

---

**Require:** Detected technique  $T_t$ , previous lifecycle  $L_{t-1}$ , system state  $Sys_t$ , action space  $U$ , RL policy  $\pi$

**Ensure:** Selected response action  $u_t^*$

- 1: Update lifecycle:  $L_t \leftarrow L_{t-1} \oplus T_t$
  - 2: Construct state:  $S_t \leftarrow \text{Encode}(L_t, Sys_t)$
  - 3: Initialize valid action set  $U_t^{valid} \leftarrow \emptyset$
  - 4: **for** each action  $u \in U$  **do**
  - 5:     **if**  $u$  is executable under the current system state  $Sys_t$  **then**
  - 6:          $U_t^{valid} \leftarrow U_t^{valid} \cup \{u\}$
  - 7: Select response action:  $u_t^* \leftarrow \pi(S_t, U_t^{valid})$
  - 8: Execute  $u_t^*$  by the corresponding AI Agent
  - 9: Observe next system state  $Sys_{t+1}$  and attack status
  - 10: **if** the lifecycle is stopped **then**
  - 11:     Assign positive reward for successful blocking
  - 12:     Add earlier-stop reward according to the current lifecycle progression
  - 13: **else if**  $u_t^*$  is a high-impact action selected too early **then**
  - 14:     Assign penalty for early aggressive response
  - 15: **else if**  $u_t^*$  is repeated and still fails to stop the attack **then**
  - 16:     Assign repeated-action penalty
  - 17: **else**
  - 18:     Assign penalty according to response delay and action cost
  - 19: Receive reward  $R_t$
  - 20: Store transition  $(S_t, u_t^*, R_t, S_{t+1})$
  - 21: Update RL policy during training
  - 22: **return**  $u_t^*$
- 

The action space  $U$  defines the set of response actions that can be selected by the RL policy. In this work,  $U$  is designed as a shared action space across all agents instead of a separate action set for each individual agent. This design allows each agent to learn a common response policy that considers not only its own executable actions, but also the possible actions of peer agents. Therefore, an agent can understand whether an observed attack state is better handled by its local capability or by another security tool with a more suitable response role. The actions are assigned according to the capability of each security tool, including firewall-level containment,

endpoint-level containment, network-flow control, packet-level interruption, and alert generation. Table ?? summarizes the response action space used in this work.

Table 4.1: Response action space

| Agent    | Action                | Description                                                                                                                                                                     | Impact    |
|----------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| All      | Alert                 | Agent issues a warning of potential attack presence, but lacks sufficient information to proceed with stronger containment actions.                                             | Low       |
| Firewall | Blocking Malicious IP | Blocks the malicious external IP address. This action is useful when the attacker source is known and can be contained at the network boundary.                                 | Low       |
|          | Stop SSH              | Disables or blocks SSH service to prevent remote access abuse, but may also interrupt legitimate remote administration.                                                         | High      |
| EDR      | Isolate Host          | Disconnects the compromised endpoint from the network while keeping the host powered on for containment and investigation.                                                      | High      |
|          | Disable Account       | Disables the affected user account, typically an administrative account in this setting, to prevent further account-based abuse.                                                | High      |
|          | Reset Password        | Forces a password reset for the compromised account. This reduces future account misuse, but may not immediately terminate an attacker who is already inside an active session. | Medium    |
|          | Kill Process          | Terminates a suspicious or malicious process on the endpoint when process-level evidence is available.                                                                          | Medium    |
|          | Shutdown              | Powers off the compromised host completely to stop ongoing malicious activity. This is the most disruptive endpoint action.                                                     | Very High |
| NDR      | Quarantine Flow       | Blocks or isolates a suspicious network flow or session identified from network behavior.                                                                                       | Medium    |
| IDS      | Drop Packet           | Drops suspicious packets based on IDS/IPS detection logic to interrupt malicious traffic at the packet level.                                                                   | Low       |

Each action is also associated with an impact level to represent its operational cost and potential disruption to normal system activity. Low-impact actions, such as issuing an alert, blocking a malicious IP address, or dropping suspicious packets, are designed for early-stage or low-confidence situations because they have limited influence on normal services. Medium-impact actions, such as resetting a password, killing a suspicious process, or quarantining a suspicious flow, provide stronger containment but may affect specific users, processes, or sessions. High-impact actions, such as isolating a host, disabling an account, or stopping SSH, can significantly interrupt system operation and are therefore more suitable when the attack has progressed further or stronger evidence is available. The Very High impact level is assigned to host shutdown, which completely stops activity on the compromised endpoint but causes the highest operational disruption.

The action space reflects the different response roles of the agents. The Firewall agent is suitable for fast boundary-level containment, the IDS agent provides low-cost packet-level interruption, the NDR agent handles suspicious network flows, and the EDR agent performs stronger endpoint-level containment. By learning over the shared action space, the RL policy can compare the expected benefit and cost of actions across different agents. As a result, the reward design encourages agents to prefer low-impact actions when they are sufficient, while reserving high-impact or very high-impact actions for later or more severe stages of the attack lifecycle.

## 4.5 Example Run

Figure 4.7 illustrates an example run of the proposed distributed AI agent-driven framework. In this example, the attack lifecycle progresses through several MITRE ATT&CK techniques, including T1007, T1016, T1027, T1049, T1057, and T1071. At the early stages, the EDR agent can detect techniques with high confidence, such as T1007, T1016, and T1027, where the confidence scores are higher than the threshold  $\theta = 0.7$ . Therefore, the agent directly accepts the detection result and sends the detected lifecycle state to the Response Module. The response agent then selects actions such as Alert or Kill Process. However, these actions are not sufficient to stop the lifecycle at these stages, so the attack continues to the next technique.

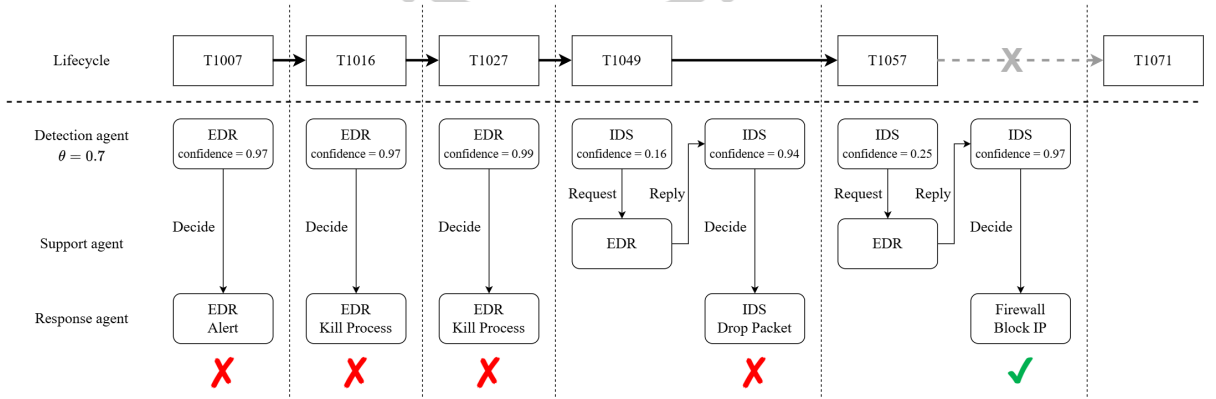


Figure 4.7: Example run of the proposed distributed detection and response process

When the IDS agent observes T1049 and T1057 with low confidence, it cannot make a confident decision by itself. Therefore, it sends a request to the EDR agent for additional contextual evidence. After receiving the reply from the EDR agent, the IDS agent updates its evidence and

obtains a higher confidence score, allowing the detection result to pass the threshold. This shows how the communication mechanism helps an agent improve detection by using evidence from peer agents instead of relying only on its local observation. Finally, at T1057, the Response Module selects the Firewall Block IP action, which successfully stops the lifecycle before the attack reaches T1071. This example demonstrates the complete workflow of the proposed solution: local detection, evidence request and reply, confidence-based decision-making, lifecycle update, and coordinated response action selection.



# Chapter 5

## Implementation

This chapter presents the practical implementation of the proposed Distributed AI Agent-driven Framework. The implementation focuses on how the proposed design is realized in a controlled experimental environment, including the overall pipeline, testbed setup, data collection, model training, and Docker-based distributed agent deployment.

### 5.1 Overall Pipeline

The overall implementation pipeline is shown in Figure 5.1. The pipeline starts from a virtualized cybersecurity testbed, where attack procedures are launched against the victim machine and observed by multiple security tools. These tools generate heterogeneous security data, including endpoint alerts, network intrusion alerts, firewall logs, and network behavior observations. The collected data is then processed and used to train the detection and response models. After training, the trained models and policies are integrated into the distributed AI Agent framework, where each agent represents one security solution and communicates with peer agents through the communication layer.

The implementation is divided into two main environments. The first environment is the VMware-based testbed used for attack simulation and security data collection. This environment includes the attacker machine, victim machine, firewall machine, and alert mapping machine. The second environment is the Docker-based distributed agent deployment environment. This environment is used to implement and evaluate the proposed multi-agent framework, including the EDR agent, IDS agent, NDR agent, Firewall agent, NATS, and Redis. By separating the data collection environment from the agent deployment environment, the implementation can generate realistic security observations while keeping the distributed agent system modular and reproducible.



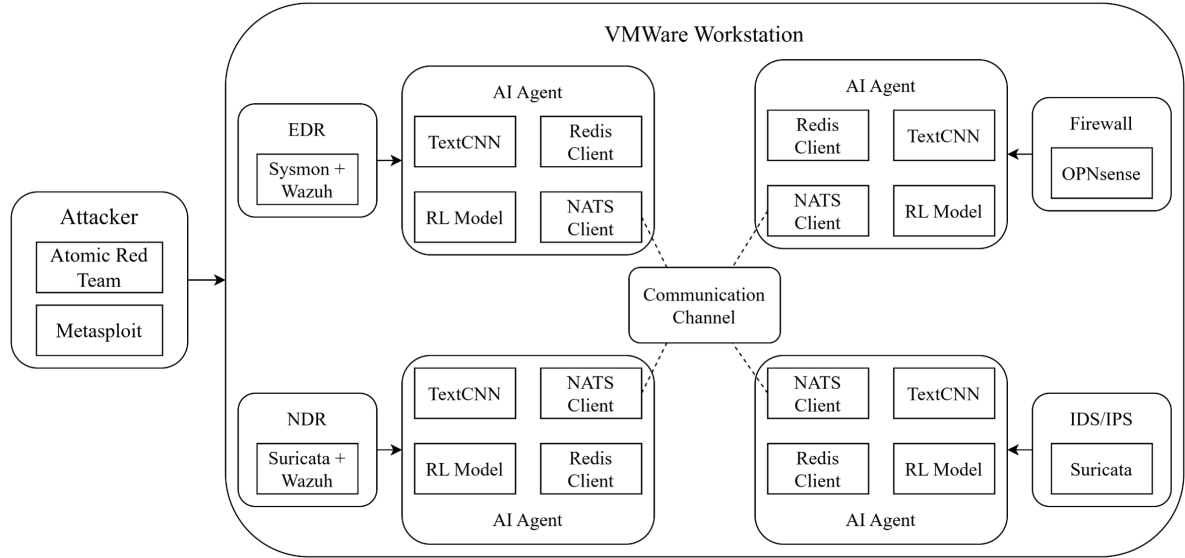


Figure 5.1: Implementation architecture

Several open-source tools and libraries are used to build the implementation environment. VMware Workstation is used to create and manage the virtualized cybersecurity testbed, where different virtual machines are assigned specific roles such as attacker, victim, firewall, and alert collection. Wazuh, Suricata, and OPNsense are deployed as the main security tools. Wazuh provides endpoint-side monitoring and alert collection, Suricata detects suspicious network activities, and OPNsense enforces firewall policies while also generating firewall-level logs.

For the distributed AI Agent framework, NATS and Redis are used to support communication and coordination among agents. NATS provides the publish/subscribe message channel, while Redis serves as a temporary blackboard for storing coordination states and recent observations. Docker is used to deploy each AI Agent as an independent container, making the framework modular and easier to reproduce. Python is used to implement the agent logic, PyTorch supports the TextCNN-based detection model, Stable-Baselines3 is used for reinforcement learning-based response selection, and Matplotlib is used to visualize experimental results. Table 5.1 summarizes the tools and libraries used in the implementation.

Table 5.1: List of tools and libraries used in the implementation

| Category                      | Tool / Library     | Description                                        |
|-------------------------------|--------------------|----------------------------------------------------|
| Environment                   | VMware Workstation | Virtualizes and manages the lab environment.       |
| Security Tools                | Wazuh              | Monitors and detects endpoint threats.             |
|                               | Suricata           | Detects and prevents network intrusions.           |
|                               | OPNsense           | Enforces network traffic policies.                 |
| Communication Layer           | NATS               | Provides the communication channel among agents.   |
|                               | Redis              | Serves as the blackboard for communication states. |
| AI Agent                      | Docker             | Deploys AI Agents in containers.                   |
|                               | Python             | Implements the AI Agent logic.                     |
|                               | PyTorch            | Provides deep learning support.                    |
|                               | Stable-Baselines3  | Provides reinforcement learning algorithms.        |
| Attack Simulation & Knowledge | Atomic Red Team    | Simulates attacks with MITRE ATT&CK.               |
|                               | Metasploit         | Simulates attacks with MITRE ATT&CK.               |
|                               | MITREtrieval       | Provides attack lifecycle knowledge.               |
| Visualization & Evaluation    | Matplotlib         | Plots metrics and experimental results.            |

## 5.2 Testbed Setup and Data Collection

The testbed is built on a physical host running Windows 11 Education with VMware Workstation. This host provides the computing resources needed to run multiple virtual machines at the same time. Table 5.2 shows the physical host configuration. The 64 GB memory allows the attacker machine, victim machine, firewall machine, alert mapping machine, and agent deployment machine to run during the implementation process.

Table 5.2: Physical host configuration

| Item                    | Configuration                                |
|-------------------------|----------------------------------------------|
| Processor               | Intel(R) Core(TM) i7-14700, 2.10 GHz         |
| Installed RAM           | 64.0 GB                                      |
| System Type             | 64-bit operating system, x64-based processor |
| Operating System        | Windows 11 Education                         |
| OS Version              | 25H2                                         |
| OS Build                | 26200.8246                                   |
| Virtualization Platform | VMware Workstation                           |

The virtualized testbed configuration is summarized in Table 5.3. The attacker machine runs Kali GNU/Linux and launches attacks using Atomic Red Team and Metasploit, while the Windows Server 2008 victim machine serves as the target host. OPNsense provides firewall control and logs, and an Ubuntu machine with Wazuh is used for alert collection and mapping without

acting as a centralized SOC decision point.

Table 5.3: Virtualized testbed configuration for data collection

| Component             | Configuration                                                  | Role                                                                           |
|-----------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------|
| Victim Machine        | Windows Server 2008; 8 GB RAM; 4 vCPUs; 250 GB disk            | Target host under attack; generates endpoint-side evidence collected by Wazuh. |
| Attacker Machine      | Kali GNU/Linux Rolling 2025.1; 8 GB RAM; 4 vCPUs; 80.1 GB disk | Launches attack procedures using Atomic Red Team and Metasploit.               |
| Firewall Machine      | FreeBSD with OPNsense; 2 GB RAM; 2 vCPUs; 128 GB disk          | Provides firewall control, traffic logging, and blocking capability.           |
| Alert Mapping Machine | Ubuntu 20.04.6 LTS with Wazuh; 12 GB RAM; 6 vCPUs; 128 GB disk | Collects and maps EDR/NDR-related alerts; not used as a centralized SOC.       |

The attack cases used in this thesis are constructed based on lifecycle knowledge derived from MITRE ATT&CK techniques. In particular, this thesis refers to the MITREtrieval study, which retrieves MITRE techniques from unstructured cyber threat intelligence reports by combining deep learning and ontology-based knowledge [35]. Based on this knowledge source, 280 attack lifecycles are selected for simulation, where each lifecycle contains 5 to 20 MITRE ATT&CK techniques. This setting provides diverse multi-stage attack sequences for evaluating detection and response across different security tools.

The data collection process follows the selected attack lifecycle setting. Each attack case is executed in the testbed and observed by the deployed security tools from different perspectives. Wazuh collects endpoint-side evidence from the victim host, Suricata generates network intrusion alerts, and OPNsense records firewall-level traffic events. The collected data is then organized by lifecycle ID and MITRE technique so that the later evaluation can measure not only whether techniques are correctly detected, but also how fast the framework can detect and respond to the attack progression.

Timing information is preserved during data collection because latency is an important part of the proposed framework. Different security tools may observe the same attack at different times. For example, a network-side alert may appear earlier, while endpoint-side evidence may arrive later but provide richer context. This difference is useful for evaluating whether distributed agents can make faster decisions by using locally available evidence and communication with peer agents.

### 5.3 Model Training and Integration

The detection and response models are trained using Google Colab before being integrated into the Docker-based agent deployment. In this implementation, Google Colab with an NVIDIA L4 GPU runtime is used to provide sufficient computing resources for model training and experiment execution without increasing the resource requirement of the local deployment machine. The Colab environment is mainly used for data preprocessing, model training, evaluation, and exporting the trained model files. After training, the TextCNN detection model and the learned reinforcement learning response policy are saved and loaded by the AI Agent containers during deployment.

The implementation mainly uses Python libraries for data processing, model training, reinforcement learning, and visualization. For the Detection Module, pandas and NumPy are used to load and preprocess alert datasets, PyTorch is used to implement and train the TextCNN model, and scikit-learn is used for dataset splitting and metric calculation. For the Response Module, Gymnasium is used to build the custom cyber response environment, while Stable-Baselines3 is used to train the reinforcement learning policy with PyTorch as the backend. Matplotlib is used in both modules to visualize training curves, detection metrics, reward trends, and response evaluation results.

### 5.4 Docker Deployment and Communication Setup

The distributed AI Agent framework is deployed on an Ubuntu Server 22.04 virtual machine using Docker. The Docker environment contains four agent containers, representing the EDR agent, IDS agent, NDR agent, and Firewall agent. It also contains two supporting service containers: NATS and Redis. Table 5.4 summarizes the Docker deployment environment used in the implementation.

Each agent container runs independently and represents the local logic of one security solution. The EDR agent handles endpoint-related evidence, the IDS agent handles intrusion de-

Table 5.4: Docker deployment environment

| Item                          | Configuration                                       |
|-------------------------------|-----------------------------------------------------|
| Deployment Operating System   | Ubuntu Server 22.04                                 |
| Memory                        | 4 GB                                                |
| Processors                    | 2 processors                                        |
| Hard Disk                     | 40 GB                                               |
| Docker Version                | 29.2.1                                              |
| Communication Bus             | NATS                                                |
| Shared Blackboard             | Redis                                               |
| Agent Containers              | EDR agent, IDS agent, NDR agent, and Firewall agent |
| Supporting Service Containers | NATS container and Redis container                  |

tection alerts, the NDR agent handles network behavior information, and the Firewall agent represents firewall-level action capability. This design allows the implementation to simulate the distributed structure of the proposed framework while keeping each component modular.

NATS and Redis provide the communication and coordination layer for the agents. NATS is used as the publish/subscribe message bus, allowing agents to exchange detection results, requests, and replies asynchronously. Redis is used as a shared blackboard to store temporary coordination states, such as recent observations and lifecycle-related information. Although NATS and Redis are shared infrastructure in the prototype, the detection and response logic remains distributed across the agent containers rather than being executed by a centralized SOC component.

The Docker deployment is used to validate the distributed behavior of the proposed framework. It allows the agents to communicate, check requests, perform leader-based coordination, and execute the response flow in a controlled and reproducible environment. This setup makes it possible to evaluate the proposed framework at the system level, including both local agent behavior and inter-agent coordination.

# Chapter 6

## Evaluation and Results

This chapter evaluates the proposed distributed AI agent-driven framework through four experimental issues. The first issue examines whether the distributed framework can achieve detection performance close to the centralized setting while reducing detection latency. The second issue analyzes the contribution of each agent in the detection process to show how different security tools provide complementary evidence. The third issue evaluates the response module by comparing the stopping behavior and response cost between centralized and distributed decision-making. The fourth issue further analyzes the response behavior of each agent to understand how different agents participate in the final action selection process. Through these four issues, the experiments demonstrate not only the detection and response effectiveness of the proposed framework, but also the different roles played by agents in distributed cyber defense.

### 6.1 Experiment Setup

For the detection experiment, the input data is constructed from security data sources. Each source contains alert text, MITRE ATT&CK technique label, and lifecycle identifier. Before training, the implementation checks whether the three sources have the same number of rows and whether their technique and lifecycle labels are consistent. The alert texts from the three sources are then merged into one input field using source prefixes such as EDR:, NDR:, and IDS:, separated by [SEP]. This design allows the model to learn from multi-source evidence while still preserving the origin of each alert.

The dataset is split according to lifecycle identifiers rather than individual alert rows. The training and testing lifecycle IDs are loaded from the `Train_Set` and `Test_Set` sheets of the split file. This split strategy prevents alerts from the same lifecycle from appearing in both training and testing sets, which makes the evaluation closer to real attack-scenario testing. Empty alert samples are removed from training, and labels that do not appear in the training set are removed from the test set. Each technique label is encoded using a label encoder, while the vocabulary is built from the training alert texts only. The main configuration of the TextCNN detection model is summarized in Table 6.1.

Table 6.1: TextCNN detection model configuration

| Item                | Configuration                                                                |
|---------------------|------------------------------------------------------------------------------|
| Input Format        | Merged alert text with source prefixes and [SEP] separator                   |
| Train/Test Split    | Split by lifecycle ID using <code>Train_Set</code> and <code>Test_Set</code> |
| Tokenizer           | Regex-based tokenizer with lowercase normalization                           |
| Vocabulary          | Built from training text only; minimum frequency = 1                         |
| Maximum Length      | 256 tokens                                                                   |
| Embedding Dimension | 128                                                                          |
| Number of Filters   | 128 filters for each filter size                                             |
| Filter Sizes        | 3, 4, and 5                                                                  |
| Dropout             | 0.2                                                                          |
| Batch Size          | 32                                                                           |
| Number of Epochs    | 15                                                                           |
| Optimizer           | Adam                                                                         |
| Learning Rate       | $5 \times 10^{-4}$                                                           |
| Loss Function       | Cross-entropy loss                                                           |
| Random Seed         | 42                                                                           |
| Model Selection     | Best checkpoint selected by macro-F1                                         |
| Evaluation Metrics  | Accuracy, macro-F1, micro-F1                                                 |

The TextCNN model consists of an embedding layer, multiple one-dimensional convolution layers, max-pooling layers, dropout, and a fully connected output layer. The convolution layers use filter sizes of 3, 4, and 5 to capture local patterns with different lengths in the alert text. After convolution and max pooling, the extracted features are concatenated and passed to the final classification layer.

For the response experiment, the response process is implemented as a custom Gymnasium environment and trained using DQN from Stable-Baselines3. Each episode represents one attack lifecycle, and the model selects actions based on the current lifecycle state and system state. The

same hyperparameters are used for both centralized and distributed settings to ensure that the comparison focuses on the framework design rather than different learning configurations. The main configuration of the reinforcement learning response model is summarized in Table 6.2.

Table 6.2: Reinforcement learning response model configuration

| Item                           | Configuration                                 |
|--------------------------------|-----------------------------------------------|
| RL Library                     | Stable-Baselines3                             |
| Environment                    | Custom Gymnasium cyber response environment   |
| Algorithm                      | DQN                                           |
| Policy                         | MlpPolicy                                     |
| Training Timesteps             | 15,000                                        |
| Random Seed                    | 42                                            |
| Learning Rate                  | $1 \times 10^{-3}$                            |
| Replay Buffer Size             | 20,000                                        |
| Learning Starts                | 300 timesteps                                 |
| Batch Size                     | 64                                            |
| Discount Factor $\gamma$       | 0.95                                          |
| Training Frequency             | Every 4 steps                                 |
| Target Network Update Interval | 750 steps                                     |
| Exploration Fraction           | 0.40                                          |
| Final Exploration Rate         | 0.05                                          |
| Compared Settings              | Centralized response and distributed response |

The response evaluation records episode reward, blocking result, stopping step, response latency, action distribution, and high-impact action ratio. These metrics are used to compare whether the distributed framework can stop attack lifecycles earlier than the centralized setting while controlling the cost of selected actions.

## 6.2 Detection: Distributed vs. Centralized

This section evaluates the detection capability of the proposed distributed AI agent-driven framework by comparing it with a centralized detection framework. The main purpose of this evaluation is to examine whether distributed coordination among multiple AI agents can preserve the detection accuracy of a centralized framework while reducing the overall detection time. In the centralized setting, evidence from different security tools is collected and processed together before making the final detection decision. In contrast, the distributed framework allows each AI agent to perform local detection and request complementary information from peer agents only when necessary. Therefore, the distributed framework is expected to reduce



the waiting time caused by full evidence collection while still maintaining sufficient contextual information for lifecycle reconstruction.

Figure 6.1 presents the detection accuracy of the distributed framework under different window sizes and thresholds, and compares it with the centralized framework. The result shows that the distributed framework can achieve near-centralized, and in the best setting equal, detection performance. Specifically, when the threshold is set to 0.7 and the window size is 1.0 second, both the distributed and centralized frameworks reach an accuracy of 0.903. This indicates that distributed multi-agent coordination does not necessarily sacrifice detection quality, even though the agents do not always wait for complete evidence from all sources.

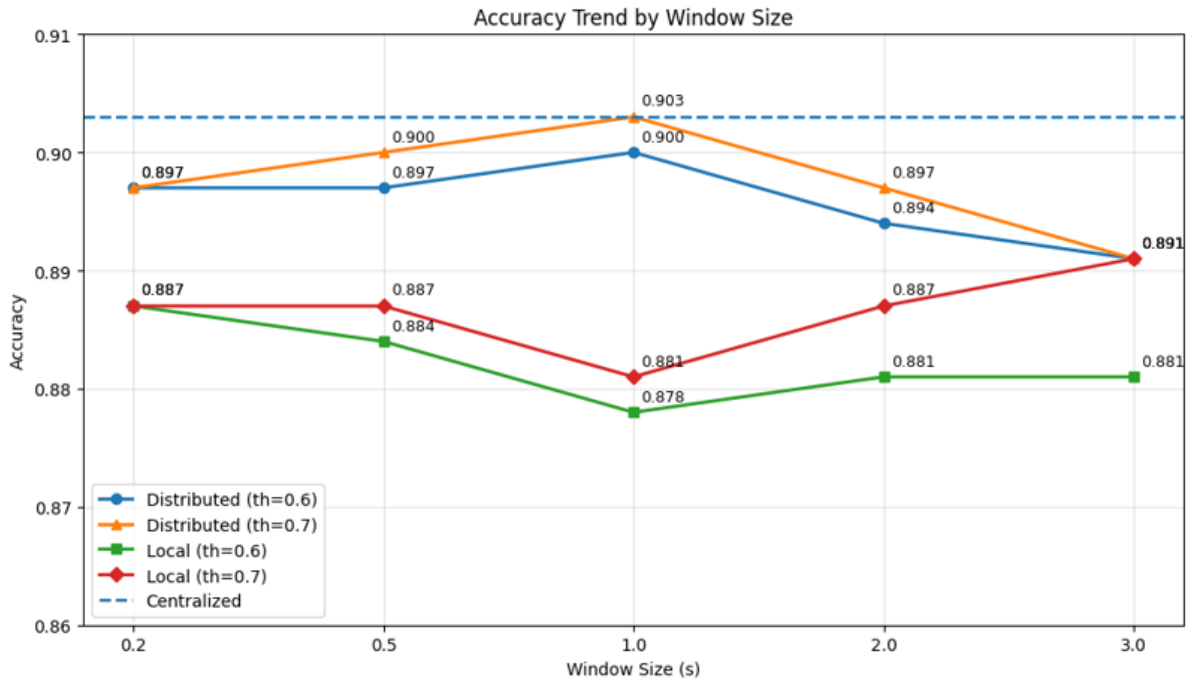


Figure 6.1: Detection accuracy comparison between distributed and centralized frameworks.

The influence of window size is also important. When the window size is smaller than 1.0 second, each agent receives less contextual information, which may limit its ability to identify techniques accurately. A 1.0-second window provides a better balance because it is long enough to collect useful peer information while still keeping the detection process responsive. When the window size becomes larger than 1.0 second, the framework may collect more local information, but the benefit in accuracy becomes less significant compared with the additional delay introduced by a longer observation window. Therefore, the 1.0-second window is selected as a practical setting for balancing detection quality and detection speed.

The threshold also affects detection performance. A threshold of 0.7 provides better overall performance in this experiment. When the threshold is too low, the models tend to produce more positive predictions, which increases the number of detected techniques but also introduces more false positives. This reduces the reliability of lifecycle reconstruction. In contrast, a properly selected threshold allows the distributed agents to preserve useful detections while avoiding excessive incorrect predictions.

Although the distributed and centralized frameworks can reach the same accuracy in the best setting, their detection time is significantly different. As shown in Figure 6.2 and Table 6.3, the distributed framework achieves much faster detection than the centralized framework. At the 1.0-second window size, the mean detection time of the distributed framework is 2.175 seconds, while the centralized framework requires 5.250 seconds. This means that the distributed framework is 58.6% faster than the centralized framework while maintaining the same accuracy of 0.903.

Table 6.3: Mean detection time under different window sizes.

| Window     | Mean Time (seconds) |              |             |
|------------|---------------------|--------------|-------------|
|            | Distributed         | Local        | Centralized |
| 0.2        | 1.527               | 1.526        |             |
| 0.5        | 1.724               | 1.723        |             |
| <b>1.0</b> | <b>2.175</b>        | <b>2.175</b> | 5.250       |
| 2.0        | 2.932               | 2.931        |             |
| 3.0        | 3.640               | 3.640        |             |

The main reason for this speed improvement is the reduction of evidence collection delay. In the centralized framework, the system must wait until sufficient evidence from multiple sources is collected before performing detection and reconstruction. This creates a long waiting period, especially when some tools generate useful evidence later than others. In contrast, the distributed framework allows an agent to start detection from its local observation and request peer information only when needed. As a result, the distributed framework avoids waiting for all evidence to become available before making progress.

The timing breakdown further supports this observation. The evidence collection delay in the distributed framework is much shorter than that of the centralized framework, while the additional runtime overhead introduced by the distributed design remains very small. The com-

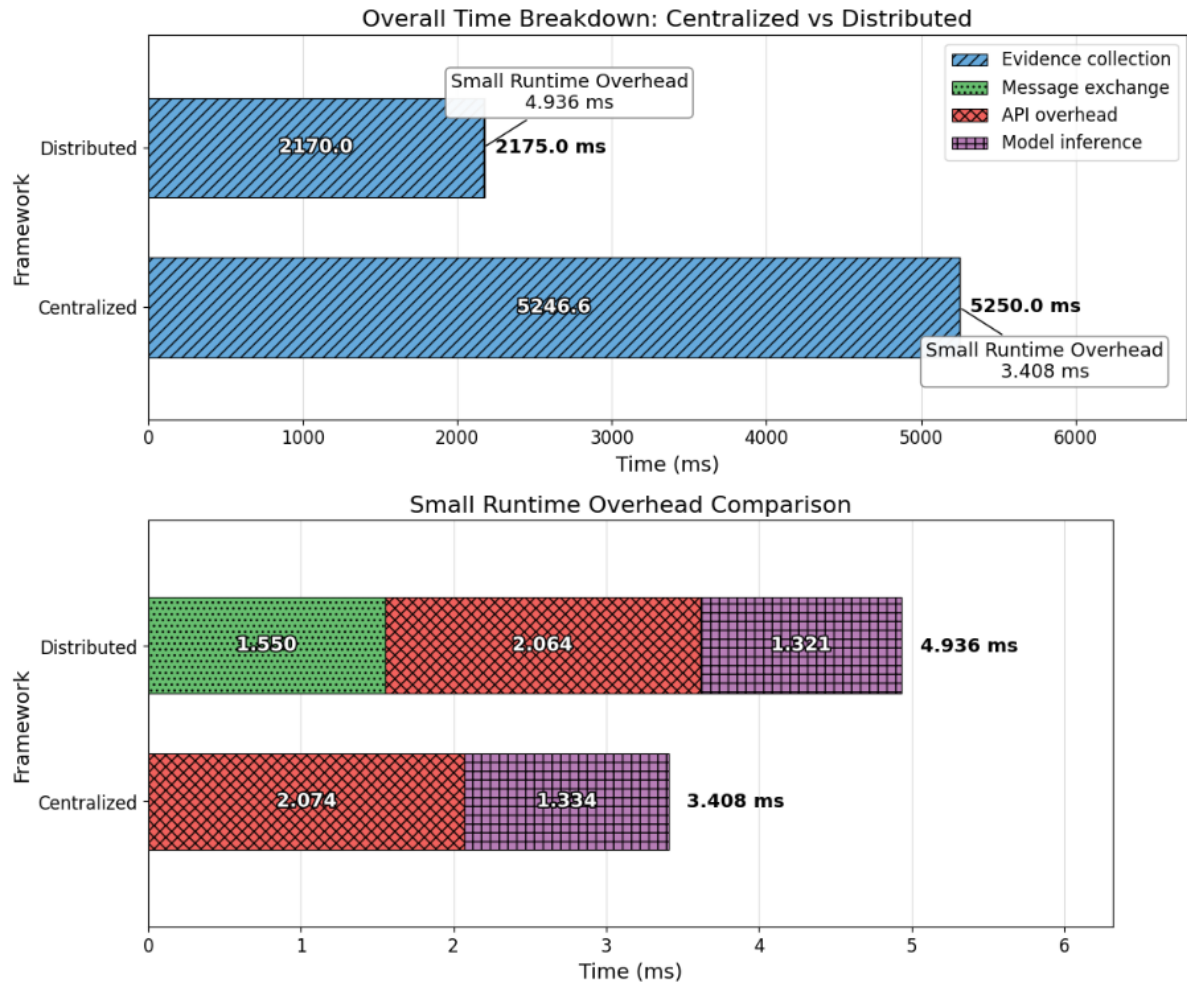


Figure 6.2: Detection time breakdown of distributed and centralized frameworks.

munication, API, and model inference overhead is only 4.936 ms, which is negligible compared with the several-second reduction in detection time. This shows that the cost of inter-agent communication is minor, and the speed gain mainly comes from avoiding unnecessary evidence waiting.

Overall, the result demonstrates that distributed AI agents can achieve detection accuracy comparable to a centralized framework while providing much faster detection. This is an important advantage for cyber defense because earlier detection allows the system to move to the response phase sooner. Therefore, the distributed framework is more suitable for time-sensitive attack scenarios, where delaying detection until all evidence is centrally collected may reduce the effectiveness of subsequent response actions.

### 6.3 Detection Coverage of Each AI Agent

This section analyzes the contribution of each AI agent in reconstructing the attack lifecycle. While the previous section shows that the distributed framework can achieve detection accuracy comparable to the centralized framework with much faster detection time, this section further investigates how this performance is formed by different agents. Since each AI agent is assigned to a specific security tool, the quality and type of information observed by each agent are different. Therefore, the purpose of this evaluation is to understand which agents provide major detection evidence, which agents play a supporting role, and how inter-agent communication helps complete the lifecycle reconstruction.

Figure 6.3 shows the contribution of each AI agent in lifecycle reconstruction. The result indicates that the EDR agent provides the largest contribution, covering 50.76% of the reconstructed lifecycle. The IDS agent is the second most important contributor, with a contribution of 39.21%. In contrast, the NDR agent contributes only 0.30%, while the Firewall agent does not directly contribute to technique reconstruction in this evaluation. This result shows that the agents do not contribute equally. Instead, each agent has a different level of usefulness depending on the semantic richness of its assigned data source.

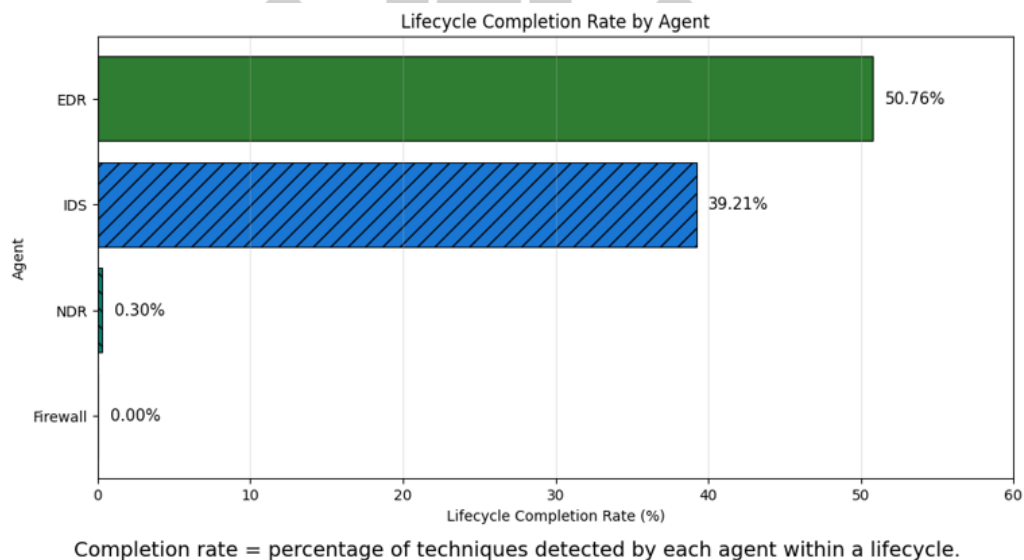


Figure 6.3: Lifecycle reconstruction contribution of each AI agent.

The dominant contribution of the EDR agent can be explained by the quality of endpoint-level evidence. Many attack techniques leave clear traces on endpoints, such as suspicious

commands, malicious processes, script execution, file operations, or privilege-related behaviors. These patterns are usually more explicit than network-level traces, making them easier for the detection model to associate with MITRE ATT&CK techniques. As a result, the EDR agent becomes the major source of reliable detection evidence in the distributed framework.

The IDS agent also provides an important contribution, but its role is different from the EDR agent. IDS data is generated from network traffic, so it can observe attack activities from a network perspective and often detects suspicious behavior earlier than endpoint-based tools. However, network-level evidence is usually less semantically rich than endpoint logs. For example, packet-level or signature-level alerts may indicate suspicious communication, scanning, or exploitation attempts, but they may not always provide enough context to identify the exact technique confidently. Therefore, the IDS agent contributes significantly to lifecycle reconstruction, but it often requires additional context from other agents to improve confidence.

Figure 6.4 further analyzes the autonomy and dependency of the agents. The result shows that the EDR agent is highly autonomous: 100% of its correct detections are produced from local EDR data. This means that when the EDR agent detects a technique correctly, it usually does not need additional evidence from peer agents. In contrast, the IDS agent is more support-dependent. In 84.5% of the cases, IDS requires support from other agents to complete or confirm its detection. This indicates that IDS is useful as an early and supportive detector, but its detection confidence is strengthened through inter-agent communication.

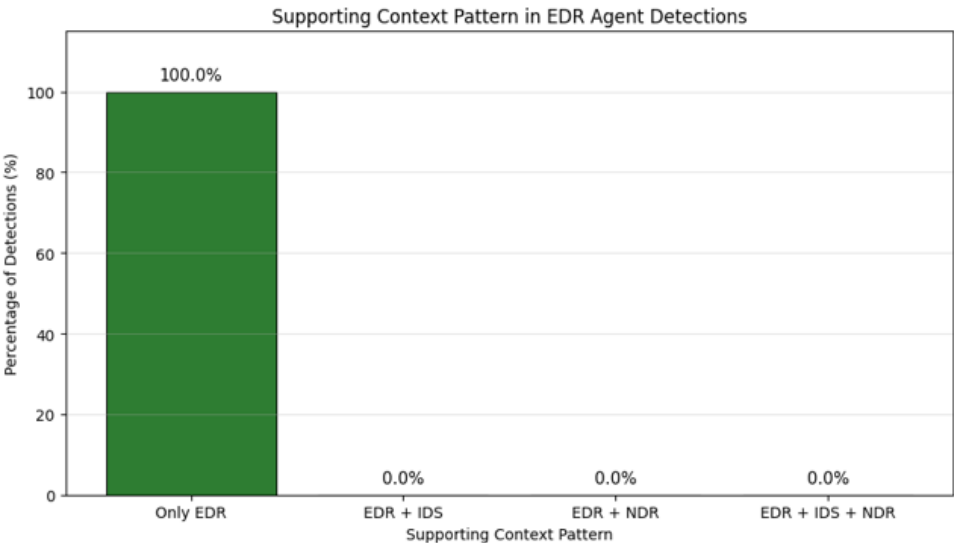


Figure 6.4: Autonomous and support-dependent detection behavior of AI agents.

The support relationship among agents is shown in Figure 6.5. The result indicates that EDR is the dominant supporting source for IDS, supporting IDS in 81.7% of the support cases. This confirms that endpoint-level evidence is not only useful for direct detection, but also valuable for validating and enriching network-level observations. In other words, IDS can provide early suspicious signals from network traffic, while EDR can provide more concrete host-level evidence to confirm the technique. This interaction explains why distributed coordination is important: the framework does not rely on each agent being equally accurate, but instead allows agents with different visibility to complement each other.

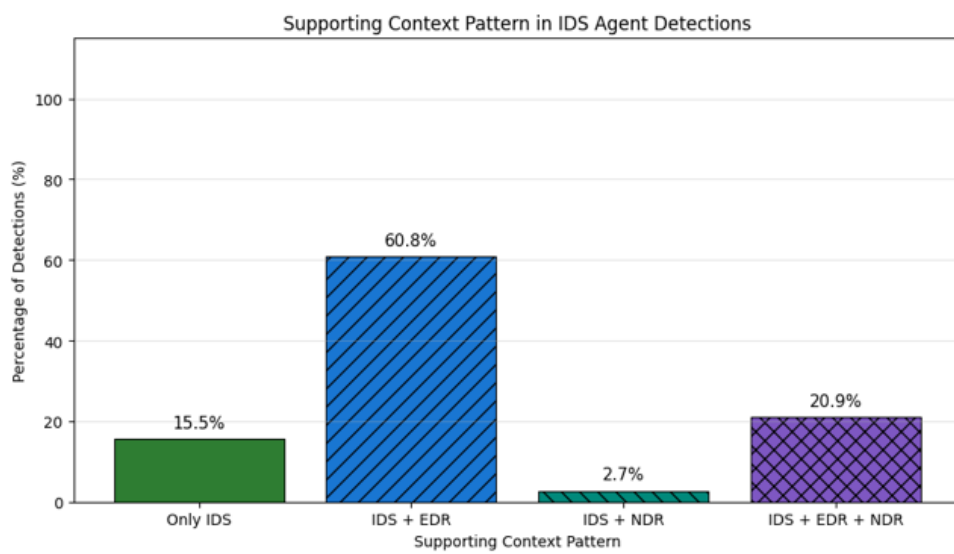


Figure 6.5: Supporting relationship among AI agents during lifecycle reconstruction.

The low contribution of the NDR and Firewall agents does not mean that these agents are unnecessary in the overall framework. Rather, it reflects the difference between detection contribution and operational utility. In this evaluation, the NDR data provides limited direct technique reconstruction because the network behavior patterns are less clear for many techniques. Similarly, firewall logs mainly describe traffic allowance or blocking decisions, which contain limited semantic information for identifying MITRE ATT&CK techniques. However, these agents can still provide useful contextual information, such as suspicious communication, malicious IP addresses, or traffic direction, which is important for correlation and response decisions.

Overall, the result shows that lifecycle reconstruction in the distributed framework is mainly driven by EDR and IDS, while other agents provide additional context depending on the available evidence. EDR acts as the major and most confident detection source, IDS acts as a fast

but support-dependent network observer, and inter-agent communication allows their strengths to be combined. Therefore, the distributed framework achieves effective lifecycle reconstruction not by making every agent equally strong, but by enabling specialized agents to contribute complementary evidence according to their own visibility.

## 6.4 Response Performance: Efficiency and Cost

This section evaluates the response performance of the proposed distributed AI agent-driven framework. The evaluation focuses on two aspects: response efficiency and response cost. Response efficiency measures whether the framework can stop an attack lifecycle earlier than a centralized framework, while response cost is analyzed through the type and timing of selected actions. The experiment is conducted over 15,000 training timesteps, where the reinforcement learning policy learns to select response actions for attack lifecycles under both distributed and centralized settings.

Figure 6.6 shows both the learning curve and the mean lifecycle stop time of the two frameworks. At the beginning of training, the centralized framework has a lower stopping step because it observes more complete information before making a response decision. In contrast, the distributed framework starts with a higher stopping step because each agent mainly relies on local observations and must learn how to coordinate its response behavior. However, as training progresses, the distributed framework improves quickly and gradually learns to stop the lifecycle earlier.

After learning, the distributed framework converges to a lower stopping step than the centralized framework. The distributed curve decreases to around 2.5 stopping steps, while the centralized curve remains around 3.5 to 4.0 stopping steps. This result indicates that distributed agents can learn to take effective actions earlier by using local evidence and shared context, instead of waiting for the complete evidence aggregation process. Although the centralized framework has richer information, its response is delayed by the need to collect and process evidence before action selection.

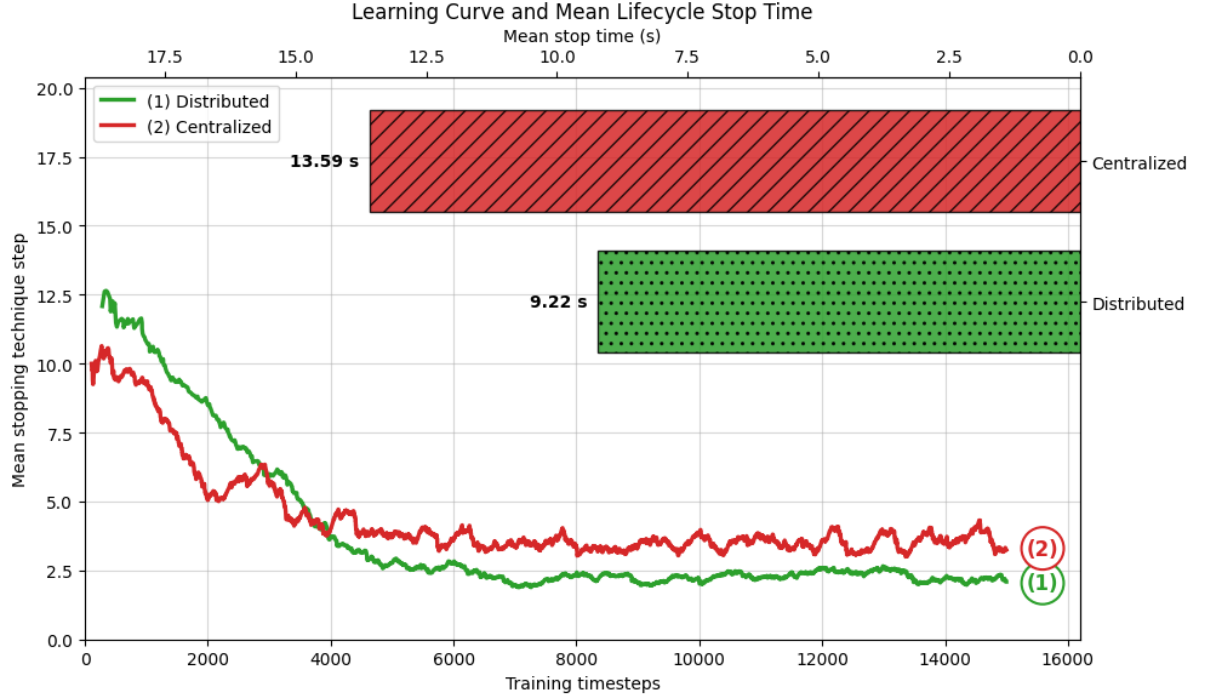


Figure 6.6: Learning curve and mean lifecycle stop time.

The upper bar plot in Figure 6.6 further compares the mean lifecycle stop time. The distributed framework stops the lifecycle in 9.22 seconds on average, while the centralized framework requires 13.60 seconds. This corresponds to a  $1.48\times$  speedup, meaning that the distributed framework is approximately 47.51% faster in response speed. In terms of response time, the distributed framework reduces the average stop time by approximately 32.21% compared with the centralized framework. This result shows that distributed decision-making can provide faster intervention in time-sensitive attack scenarios.

The main reason for this improvement is that the distributed framework allows agents to respond closer to the point where evidence is generated. In the centralized framework, the response decision is postponed because evidence from multiple sources must be collected and processed before selecting an action. In contrast, the distributed framework allows agents to act earlier when local observations and peer-shared context are sufficient. Therefore, the response process becomes more proactive and less dependent on centralized evidence waiting.



## 6.5 Response Behavior of Each Agent

The previous evaluation shows the overall response performance of the centralized and distributed frameworks. However, the average stopping step alone does not explain which agent contributes more to the response process. Therefore, this section further analyzes the selected actions in the distributed framework. The purpose is to observe how different agents participate in response decision-making and whether their response roles are consistent with their security capabilities.

Table 6.4 summarizes the average step at which each response action is selected by the learned policy. The result shows a clear ordering between low-impact and high-impact actions. The Firewall agent selects *Blocking IP* at step 2.37, which is the earliest response action. This action is relatively safe because it targets the malicious external IP and does not directly interrupt the internal host. NDR selects *Quarantine Flow* at step 4.40, which provides medium-impact containment at the network-flow level. In contrast, stronger endpoint actions such as *Disable Account*, *Isolate Host*, and *Shutdown* are selected at later stages.

Table 6.4: Average selected step of response actions.

| Action          | Agent    | Average Step |
|-----------------|----------|--------------|
| Blocking IP     | Firewall | 2.37         |
| Quarantine Flow | NDR      | 4.40         |
| Kill Process    | EDR      | 8.18         |
| Reset Password  | EDR      | 8.98         |
| Drop Packet     | IDS      | 9.83         |
| Disable Account | EDR      | 10.84        |
| Isolate Host    | EDR      | 11.14        |
| Stop SSH        | Firewall | 12.47        |
| Shutdown        | EDR      | 14.10        |

This action distribution indicates that the learned policy does not simply rely on aggressive actions to stop attacks early. Instead, it learns to use low-impact actions first when they are available and effective. For example, *Blocking IP* is selected early because it can contain external malicious traffic with relatively low operational cost. On the other hand, actions such

as *Shutdown* and *Isolate Host* are delayed because they can strongly stop the attack but also cause greater disruption to the system. This behavior supports the objective of balancing attack-blocking effectiveness and response cost.

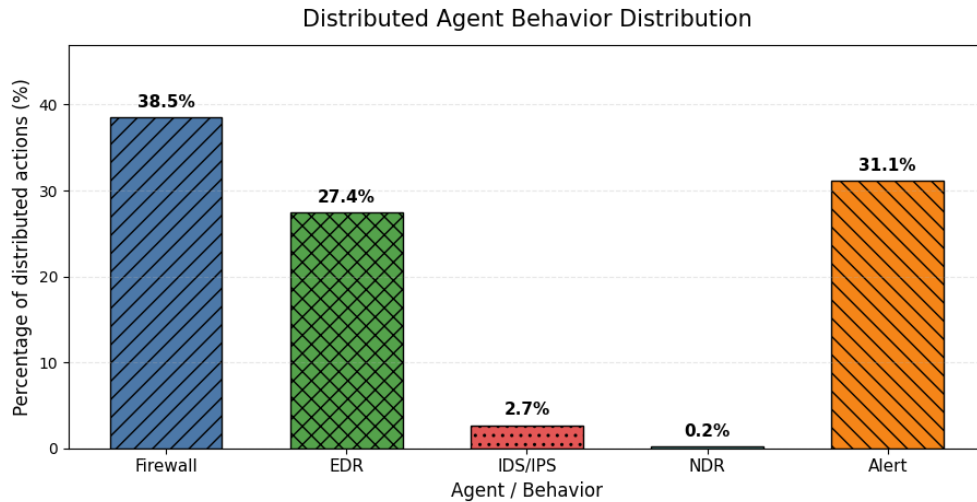


Figure 6.7: Distribution of selected response behaviors in the distributed framework

Figure 6.7 shows the distribution of selected response behaviors in the distributed framework. The Firewall agent accounts for the highest proportion of selected actions, with 38.5%, even though it does not contribute strongly to technique detection compared with agents such as EDR. This indicates that detection contribution and response contribution are not always the same: an agent may observe fewer techniques but still play an important role when its response capability is effective. In this case, the Firewall is useful for early network-level containment once malicious IP or connection information is available. The Alert behavior also takes a large proportion, with 31.1%, showing that the framework does not always execute active containment immediately and can remain cautious when stronger actions are not justified. EDR contributes 27.4%, reflecting its role in endpoint-level containment when host, account, or process evidence is available. In contrast, IDS/IPS and NDR account for only 2.7% and 0.2%, respectively, suggesting that their main value in this experiment is more related to detection and evidence sharing than direct response execution. Overall, this result highlights that different agents have different roles in the distributed framework, where some agents mainly provide detection context while others are more important for executing response actions.

# Chapter 7

## Conclusion and Future Work

This thesis proposed a distributed AI agent-driven framework for coordinated threat detection and response across heterogeneous security tools, including EDR, IDS, NDR, and Firewall. The main problem addressed in this work is that modern security evidence is fragmented across multiple tools, while centralized processing requires additional time to collect, correlate, and interpret alerts before making decisions. To reduce this delay, the proposed framework distributes intelligence to specialized AI Agents located closer to the data sources.

The framework consists of three main modules. The Communication Module uses a leader mechanism to coordinate collaboration among agents and avoid duplicated processing. The Detection Module uses a lightweight TextCNN model to classify structured and semi-structured alerts into MITRE ATT&CK techniques. The Response Module uses reinforcement learning to select defensive actions based on the reconstructed lifecycle and the current system state. Through this design, each agent can preserve local specialization while still contributing to coordinated detection and response.

The evaluation shows that the distributed framework achieves detection performance comparable to the centralized framework. In the best setting, with a threshold of 0.7 and a window size of 1.0 second, both frameworks reach an accuracy of 0.903. However, the distributed framework requires only 2.175 seconds on average, compared with 5.250 seconds for the centralized framework. This corresponds to a 58.6% reduction in detection time, mainly due to lower evidence collection delay.

The agent contribution analysis shows that agents play different roles in lifecycle reconstruction. EDR contributes the most with 50.76% lifecycle coverage because endpoint evidence contains rich information such as commands, processes, and host activities. IDS also contributes 39.21%, but it often requires support from other agents; in 84.5% of its correct detections, IDS

depends on additional information, with EDR being the dominant supporting source. NDR and Firewall contribute less directly to technique reconstruction, but they still provide useful operational context such as suspicious flows and malicious IP information.

For response performance, reinforcement learning enables the distributed framework to stop attack lifecycles earlier than the centralized framework. After learning, the distributed framework stops lifecycles around step 2.5, while the centralized framework remains around step 3.5 to 4.0. The average response time is reduced from 13.60 seconds to 9.22 seconds, giving a  $1.48\times$  speedup, or approximately 32.21% lower response time. The learned policy also shows cost-aware behavior, selecting low-impact actions such as *Blocking IP* earlier and delaying high-impact actions such as *Isolate Host*, *Stop SSH*, and *Shutdown* to later stages.

Overall, the results demonstrate that distributed AI Agents can maintain near-centralized detection accuracy while improving both detection and response speed. The framework reduces unnecessary waiting time, preserves tool-specific visibility, and enables earlier intervention during attack progression. These findings support the feasibility of moving part of the decision intelligence from a centralized component to multiple collaborative agents.

For future work, the framework can be extended in three directions. First, the evaluation can be expanded with larger and more realistic datasets, including benign background traffic, noisy alerts, and more attack families. Second, the system can be improved toward real-time deployment by connecting agents to live streaming logs and strengthening communication against delayed or missing messages. Third, the response policy can be refined with stronger safety constraints, such as asset importance, business impact, rollback mechanisms, and human approval for high-impact actions.

# References

- [1] S. Ndichu, T. Ban, T. Takahashi, and D. Inoue, “Ai-assisted security alert data analysis with imbalanced learning methods,” *Applied Sciences*, vol. 13, no. 3, 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/3/1977>
- [2] A. R. Muhammad, P. Sukarno, and A. A. Wardana, “Integrated security information and event management (siem) with intrusion detection system (ids) for live analysis based on machine learning,” pp. 1406–1415, 2023, 4th International Conference on Industry 4.0 and Smart Manufacturing. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050922024243>
- [3] T. Ban, T. Takahashi, S. Ndichu, and D. Inoue, “Breaking alert fatigue: Ai-assisted siem framework for effective incident response,” *Applied Sciences*, vol. 13, no. 11, p. 6610, 2023. [Online]. Available: <https://doi.org/10.3390/app13116610>
- [4] E. Iturbe, A. Rego, O. Llorente-Vazquez, E. Rios, C. Dalamagkas, D. Merkouris, and N. Toledo, “Reinforcement learning in action: Powering intelligent intrusion responses to advanced cyber threats in realistic scenarios,” *Expert Systems with Applications*, vol. 296, p. 129168, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S095741742502785X>
- [5] K. R. Ismail, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, “Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic artificial intelligence,” *Information*, vol. 16, no. 5, p. 365, 2025. [Online]. Available: <https://doi.org/10.3390/info16050365>
- [6] Gartner, “Gartner identifies the top cybersecurity trends for 2024,” <https://www.gartner.com/en/newsroom/press-releases/2024-02-22-gartner-identifies-top-cybersecurity-trends-for-2024>, Feb. 2024.
- [7] Gartner, “Quick answer: How is cybersecurity mesh applicable for midsize enterprises?” <https://www.gartner.com/en/documents/5892043>, Nov. 2024.

- [8] S. R. Pokhrel, G. Li, R. Doss, and S. Nepal, "Toward decentralized operationalization of zero trust architecture for next generation networks," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 6, pp. 1998–2010, 2025.
- [9] F. Louati, F. B. Ktata, and I. Amous, "Big-ids: A decentralized multi agent reinforcement learning approach for distributed intrusion detection in big data networks," *Cluster Computing*, vol. 27, pp. 6823–6841, 2024.
- [10] MITRE, "MITRE ATT&CK," <https://attack.mitre.org/>, 2026.
- [11] Microsoft, "Microsoft defender for endpoint," <https://www.microsoft.com/en-us/security/business/endpoint-security/microsoft-defender-endpoint>, 2026, accessed: 2026-05-28.
- [12] CrowdStrike, "CrowdStrike falcon insight xdr," <https://www.crowdstrike.com/en-us/platform/endpoint-security/falcon-insight-xdr/>, 2026, accessed: 2026-05-28.
- [13] Wazuh, "Wazuh: Open source xdr and siem," <https://wazuh.com/>, 2026, accessed: 2026-05-28.
- [14] Darktrace, "Darktrace network," <https://www.darktrace.com/products/network>, 2026, accessed: 2026-05-28.
- [15] Vectra AI, "Network detection and response," <https://www.vectra.ai/topics/network-detection-and-response>, 2026, accessed: 2026-05-28.
- [16] Zeek, "The zeek network security monitor," <https://zeek.org/>, 2026, accessed: 2026-05-28.
- [17] Cisco Talos, "Snort: Network intrusion detection and prevention system," <https://www.snort.org/>, 2026, accessed: 2026-05-28.
- [18] Open Information Security Foundation, "Suricata," <https://suricata.io/>, 2026, accessed: 2026-05-28.
- [19] Fortinet, "Fortigate next-generation firewall," <https://www.fortinet.com/products/next-generation-firewall>, 2026, accessed: 2026-05-28.

- [20] Netgate, “pfsense: Open source firewall,” <https://www.pfsense.org/>, 2026, accessed: 2026-05-28.
- [21] OPNsense, “Open source firewall and routing platform,” <https://opnsense.org/>, 2026, accessed: 2026-05-28.
- [22] Splunk, “Splunk enterprise security,” [https://www.splunk.com/en\\_us/products/enterprise-security.html](https://www.splunk.com/en_us/products/enterprise-security.html), n.d., accessed: 2026-05-28.
- [23] Palo Alto Networks, “Cortex xsoar,” <https://www.paloaltonetworks.com/cortex/cortex-xsoar>, n.d., accessed: 2026-05-28.
- [24] Shuffle, “Shuffle automation: An open source soar solution,” <https://shuffler.io/>, n.d., accessed: 2026-05-28.
- [25] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1285–1298.
- [26] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [27] Y. Kim, “Convolutional neural networks for sentence classification,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1746–1751.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [29] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

- [30] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," *arXiv preprint arXiv:2405.04760*, 2024.
- [31] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [32] S. Dong, Y. Xia, and T. Peng, "Network abnormal traffic detection model based on semi-supervised deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4197–4212, 2021.
- [33] R. Vast, S. Sawant, A. Thorbole, and V. Badgujar, "Artificial intelligence based security orchestration, automation and response system," in *2021 6th International Conference for Convergence in Technology (I2CT)*, 2021.
- [34] U. Bartwal, S. Mukhopadhyay, R. Negi, and S. Shukla, "Security orchestration, automation, and response engine for deployment of behavioural honeypots," in *2022 IEEE Conference on Dependable and Secure Computing (DSC)*, 2022, pp. 1–8.
- [35] Y.-T. Huang, R. Vaitheeshwari, M.-C. Chen, Y.-D. Lin, R.-H. Hwang, P.-C. Lin, Y.-C. Lai, E. H.-K. Wu, C.-H. Chen, Z.-J. Liao, and C.-K. Chen, "MITREtrieval: Retrieving MITRE techniques from unstructured threat reports by fusion of deep learning and ontology," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4871–4887, 2024.